

Udviklingen af 'Skraldehelten'

Analyse og dokumentation for udviklingen af et læringsspil

BSc i Software Engineering, Efterår 2025

Semester: Første semester

Fag: Semesterprojekt: Udvikling af Softwareprogrammer

Kursuskode: SI1-PRO

Projektperiode: September 2025 - Januar 2026

Forfattere:

Aliza	—
Mikkel Blom	—
Octavian	—
Yaser	—
Ziad	—

Gruppenr.: 4.4

Vejleder: Jeppe

Uddannelsesinstitution: Syddansk Universitet
Institut: Mærsk Mc-Kinney Møller Instituttet
Fakultet: Det Tekniske Fakultet

Antal anslag (inkl. mellemrum): 78.955

25. september 2026

Resumé

Dette projekt er udarbejdet som et 1. semesterprojekt på Software Engineering-uddannelsen. Rapporten tager udgangspunkt i FN's Verdensmål nr. 12 om ansvarligt forbrug og produktion, med specifikt fokus på delmål 12.4, 12.5 og 12.8. Projektets problemstilling adresserer en identificeret mangel på viden om korrekt affaldssortering hos børn og unge i alderen 10–19 år.

Som løsning er der udviklet et læringsspil, der bygger videre på det udleverede framework World of Zuul. Spillet er designet som et Tetris-lignende spil, hvor spilleren skal sortere affald korrekt på tværs af seks levels. Spillets pædagogiske fundament er baseret på trial & error-princippet, hvilket giver spilleren mulighed for at lære af sine fejl gennem feedback undervejs i spillet.

Udviklingsprocessen har været iterativ og har gennemgået flere faser. I analysen og designet af programmet er der anvendt metoder som noun/verb-analyse, CRC-kort samt UML-klassediagrammer til at strukturere løsningen. I #1 Iteration blev et MVP (Minimum Viable Product) udviklet, som efterfølgende blev videreudviklet og forbedret i #2 Iteration. En central forbedring her var implementeringen af en lagdelt softwarearkitektur, hvor systemet blev opdelt i et præsentations-, domæne- og datalag for at øge kodens kvalitet og struktur.

Afslutningsvis blev projektet testet på personer indenfor målgruppen. Gennem testen blev der identificeret flere forbedringspunkter, men ultimativt besvares problemstillingen med et nej. Trods spillets potentiale var det ikke muligt indenfor projektets rammer at påvise en reel læringseffekt hos målgruppen ved brug af spillet.

Indhold

1	Indledning	5
1.1	Projektets rammer	5
1.2	Problemanalyse	5
1.2.1	Hvad er problemet?	5
1.2.2	Problemtræ	6
1.3	Målgruppeanalyse	7
1.4	Aktøranalyse	7
1.4.1	Primære aktører	7
1.4.2	Sekundære aktører	8
1.4.3	Tertiære aktører	8
1.5	Redegørelse for 'World Of Zuul'-koden	8
1.6	Problemformulering	8
1.7	Metoder	9
1.8	Tidsplan	9
2	Krav	10
2.1	#1 Iteration	10
2.1.1	Spilbeskrivelse	10
2.1.2	Noun/Verb analyse	11
2.1.3	CRC-Kort	12
2.1.4	Kravformulering	13
2.2	#2 Iteration	15
2.2.1	Krav udvidelser	15
2.3	Brug af krav	15
3	Design	15
3.1	#1 Iteration	15
3.2	UML-klassediagram	15
3.3	#2 Iteration	17
3.4	Design beskrivelse	19
3.4.1	Softwarearkitektur	19
3.4.2	Brugergrænseflade	20
4	Implementering	22
4.1	#1 Iteration	22
4.1.1	Implementering af funktionelle krav	23
4.1.2	Implementering af non-funktionelle krav	27
4.2	#2 Iteration	27
5	Test af koden - NUnit tests	32
6	Test af løsning	36
7	Evaluerings	37
7.1	Iteration 1	37
7.2	#2 Iteration	38
8	Konklusion	38
9	Procesevaluering	38
9.1	Etablering af samarbejdsgrundlag	38
9.2	Arbejdsform og metoder	38
9.3	Kommunikation og koordinering	39
9.4	Udviklingsværktøjer og teknisk samarbejde	39
9.5	Dokumentationsværktøjer og andre værktøjer	39
9.6	Tidsstyring og arbejdsfordeling	39

9.7	Læringsprocessen	39
9.8	Refleksion over samarbejdet	39
9.9	Forbedringspunkter og erfaringer til fremtidige projekter	40
9.10	Brug af generativ AI-værktøjer	40
10	Bilag	41
10.1	Samarbejdsaftale	42
10.2	Vejlederkontrakt	49
10.3	Gruppelog	54
10.4	Noun/Verb Analyse	82
10.5	Samlede CRC-Kort	86
10.6	Kravspecifikationer	89
10.7	Kontrolskema for Rapport	91

1 Indledning

1.1 Projektets rammer

Dette projekt er 1. semesterprojekt for software engineering på SDU, 2025/26. Projektet er opstillet som et frit projekt, hvori der skal udarbejdes et spil der adresserer en problemstilling inden for en underkategori af FN's verdensmål. Herudover skal produktet bygges på det udleverede framework World of Zuul (WoZ), samt implementere udvidelser og tilføjelser til frameworket.

Dette projekt tager udgangspunkt i FN's verdensmål nr. 12, der omhandler "Ansvarligt forbrug og produktion". Yderligere har vi afgrænset os til delmål 12.4, 12.5 og 12.8 af FN's verdensmål, grundet projektets tidsmæssige rammer. For yderligere at fokusere har vi valgt at arbejde med affaldssortering og de konsekvenser, forkert sortering medfører, da dette medvirker til at opnå verdensmålet.

Fejlsortering betyder, at værdifulde ressourcer går tabt, genanvendelsesprocenten reduceres, og samfundet påføres både økonomiske og miljømæssige omkostninger. Problemet er altså ikke kun teknisk, men også adfærdsmæssigt, og det gør fejlsortering til en væsentlig barriere for en mere bæredygtig affaldshåndtering.

1.2 Problemanalyse

1.2.1 Hvad er problemet?

Danskerne sorterer mere affald til genanvendelse end tidligere. I 2023 blev 59% af husholdningsaffaldet indsamlet til genanvendelse [1]. Alligevel viser undersøgelser, at fejlsortering er meget udbredt. I en rundspørge i 2021 svarede 42% af danskerne, at de i løbet af den sidste måned har smidt noget affald i den forkerte beholder [2]. Tilsvarende fandt en anden undersøgelse i 2022, at 31% af danskerne mener, at det er "for besværligt" at sortere affald [3], og knap hver tredje borger har samtidig mistillid til, at det sorterede affald bliver genanvendt korrekt [3]. Undersøgelserne viser altså, at fejlsortering ikke kun sker som små uheld en gang imellem, men faktisk bliver gjort af en betydelig andel hvert år.

År	Genanvendelse (%)	Forbrænding (%)
2011	38	62
2022	55	45
2023	59	41

Tabel 1: Fordeling af husholdningsaffald: Genanvendelse vs. forbrænding [1]

Udviklingen i Tabel 1 viser en stigende genanvendelsesandel, fra 38% i 2011 til 59% i 2023, men samtidig illustrerer den, at en væsentlig del af affaldet stadig ikke genanvendes (41% i 2023). Der kan peges på flere forskellige årsager til fejlsortering.

Barrierer for korrekt sortering

Mange oplever, at de nye fraktioner er svære at overskue [4]. Forvirring om, hvilke materialer der hører til i hvilke beholdere, fører til, at affaldet ofte sorteres forkert.

Borgerne efterspørger mere konkret information. En spørgeskemaundersøgelse viste, at mange gerne vil vide, om "affaldet rent faktisk genanvendes" og især har brug for vejledning i sortering af komplicerede blandinger (plastik, emballage med madrester m.v.) [5]. Ifølge en nudging-ekspert nævnes netop "manglende viden om sortering" som en nøglebarriere [6].

En del borgere har svært ved at se formålet, hvis de ikke tror på, at deres sortering anvendes hensigtsmæssigt. I én undersøgelse svarede 12%, at de ikke stoler på, at deres sorterede affald håndteres miljøvenligt [7]. Det skaber usikkerhed og risiko for, at affaldet blot ender i restaffaldet.

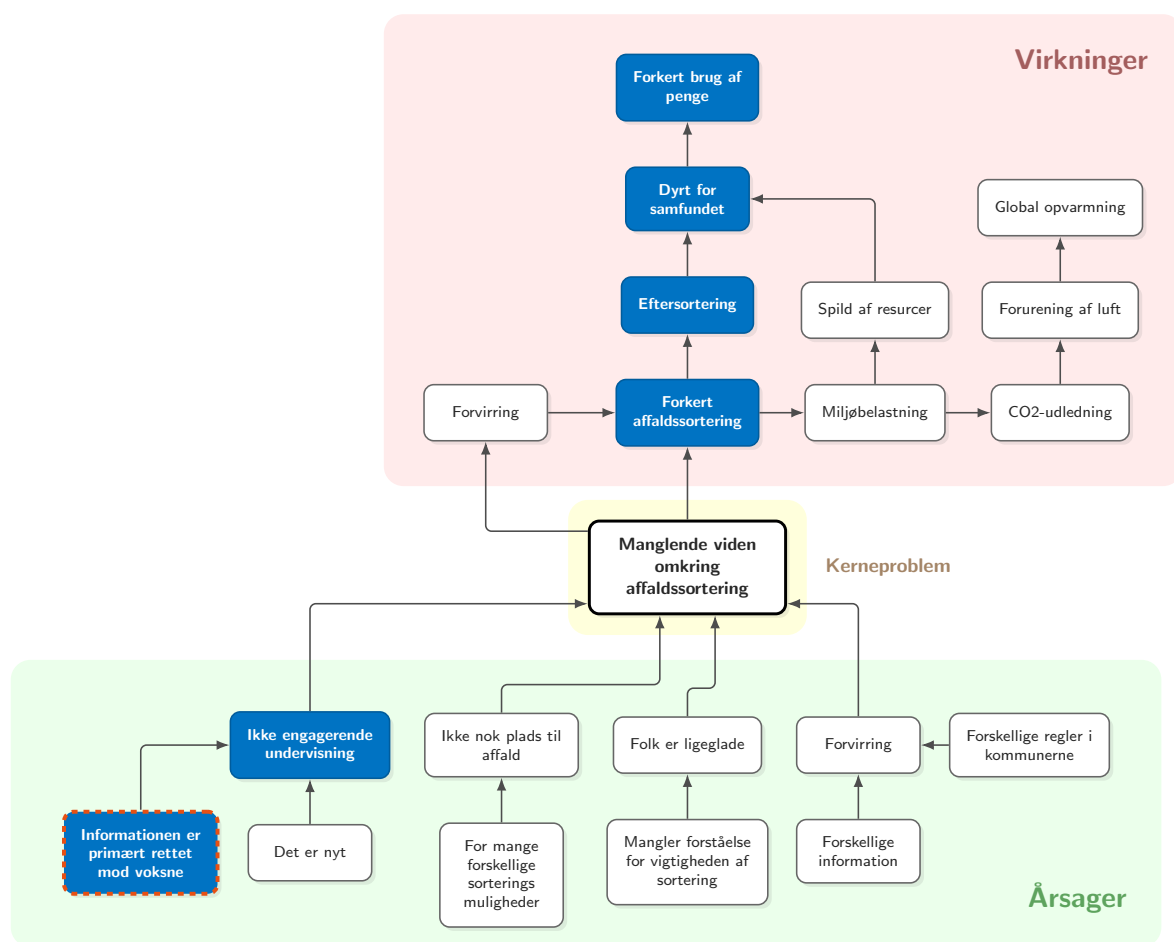
Konsekvenser og behov for information

Mangel på viden og klare retningslinjer har direkte konsekvenser for genanvendelsen. Når borgere

er i tvivl, ender ofte værdifulde materialer i forbrændingen eller kontaminerer genbrugsfraktionerne. Ifølge Plastic Change viser den nye statistik tydeligt, at danskernes viden om affaldsstrømmen er utilstrækkelig [8]. For at øge den korrekte sortering peger eksperter og forskning derfor entydigt på behovet for bedre information og uddannelse: Jo mere borgerne ved om hvorfor og hvordan affaldet genanvendes, desto mere villige og i stand er de til at sortere rigtigt [4, 5]. Sammenfattende peger alle kilder på, at det er netop manglen på viden og ikke manglende vilje, der bremser affaldssorteringen. Klar og tilgængelig information om sorteringsregler, affaldstyper og genanvendelsesproces er derfor afgørende for at mindske fejlsorteringen og sikre en høj genanvendelsesgrad.

1.2.2 Problemtræ

For at analysere problemet har gruppen lavet et problemtræ med fokus på forkert affaldssortering og har fundet de forskellige årsager og virkninger. Yderligere indsnævres problemet til den manglende viden om korrekt sortering. Dette blev defineret som vores kerneproblem, og på Figur 1 ses problemtræet, med de årsag-virknings kæder, der blev identificeret. Yderligere viser problemtræet projektgruppens afgrænsning, markeret med blå.



Figur 1: Problemtræ fokuseret på kerneproblemet ”Manglende viden om affaldssortering”, som viser den afgrænset årsag-virknings kæde, projektet tager udgangspunkt i.

Ud fra dette problem gennemgik gruppen en iterativ brainstormproces, hvor der først blev identificeret årsager til dette problem. Herefter blev virkningerne af problemet identificeret.

Årsager

En central årsag er forvirringen forbundet med affaldssystemet – et komplekst, omfattende og relativt nyt system gør også, at megen tilgængelig information er forskellig på tværs af hjemmesider, plakater

og andre informationsmedier. Derudover har flere kommuner også deres egne regler og anbefalinger om affaldssortering. Denne grundlæggende forvirring er med til at skabe den mangel på viden der er.

En anden årsag er folks villighed til at deltage – at nogen faktisk er ligeglade med at sortere deres skrald. Flere faktorer kan ligge grund for dette. Manglende forståelse gør det svært at sætte sig ind i selv at gøre det. Yderligere har nogle måske heller ikke muligheden for at tage sig af alle de affaldssorteringsbehov, der er opstået. Når man nu skal sortere rest, mad, pap, metal og plast er det lige pludselig meget plads man skal afsætte til skraldespande eller bunker til sortering.

Derudover er der også den årsag, at det er et nyt system, så den mængde information og undervisning, der er om emnet, er ikke udviklet nok, til f.eks. at engagere børn og unge i at interessere sig for emnet. Derudover formodes det også, at meget af den information der er om affaldssortering, er rettet mod den voksne befolkning. Dette efterlader en generation, der potentielt kommer til at mangle viden.

Virkninger

En central virkning er forkert affaldssortering: Hvis man ikke ved, hvordan man skal sortere sit affald, er det svært at sortere korrekt. Ligeledes kan manglen på viden skabe yderligere forvirring, der igen kan lede til at skrald sorteres forkert.

Denne centrale virkning grener sig ud og berører miljøkonsekvenserne af problemet. Forkert affaldssortering vil dermed føre til yderligere miljøbelastning, spild af ressourcer og i sidste ende også lede til en forværring af global opvarmning gennem forurening og CO₂-udledning.

Parallelt med de miljømæssige konsekvenser er der også en økonomisk virkning. Et behov for manuel eftersortering af skraldet bliver nødvendigt. Dette er en dyr proces, som tager både penge og ressourcer fra andre vigtige velfærdsområder.

Samlet set illustrerer virkningerne på problemtræet, hvordan et lokalt orienteret problem kan have store, globale konsekvenser, og er i direkte modstrid med FN's verdensmål.

1.3 Målgruppeanalyse

Børn og unge lærer som udgangspunkt bedst gennem aktive og erfaringsbaserede læringsformer, hvor de selv deltager og lærer gennem handling (learning-by-doing) [9, 10]. Et interaktivt spil er derfor et velegnet produkt for denne aldersgruppe.

Selvom børn og unge udvikler abstrakt tænkning, understøttes forståelsen fortsat af visuelle elementer og konkrete eksempler, særligt ved komplekse emner som affaldssortering [9, 10]. Samtidig motiveres unge i høj grad af social interaktion, konkurrence og anerkendelse, hvilket kan understøttes gennem point og belønninger i spillet [11, 12].

Unge har desuden et naturligt behov for udforskning og selvstyret læring, hvor de lærer gennem afprøvning og fejl [12]. En tydelig progression med gradvist stigende sværhedsgrad skaber en følelse af mestring og fastholder motivationen [12, 13]. Hurtig og tydelig feedback er ligeledes afgørende for at bekræfte spillerens indsats og opretholde engagementet [13].

Børn og unge har flere styrker, der gavner spillets formål. De har høj digital dannelse og er fortrolige med spilmekanikker og digitale interfaces [9], er modtagelige for vanedannelse i en tidlig alder [12] og besidder en voksende evne til at forstå abstrakte sammenhænge og langsigtede konsekvenser, såsom miljøpåvirkning af affaldssortering [14].

På baggrund af overstående kilder kan målgruppen defineres som de 10–19-årige. Derudover er lærings-spilstilgange særligt velegnede til dem, da deres læringsstil, motivation og digitale kompetencer understøtter en spilbaseret og engagerende tilgang.

1.4 Aktøranalyse

1.4.1 Primære aktører

Elever og studerende

Skoleelever og studerende er de to primære brugere af læringsspillet. Undervisningen i affaldssortering

på skolen kan styrke unge menneskers holdninger og vaner i forhold til sortering, både i skolen og hjemme.

Forældre

Forældrene til de børn og unge i målgruppen, der påvirkes af læringsspillet, er også en relevant aktør at overveje. Deres daglige gang vil blive påvirket af deres børns nye viden. Børn, der bliver mere opmærksomme på fejlsortering, kan bidrage til, at hele husholdningens sortering vil forbedres. Ydermere kan der forekomme vidensdeling mellem børn og forældre, som kan lede til fortsat forbedring i sortering.

1.4.2 Sekundære aktører

Kommuner

Kommunerne har det overordnede ansvar for affaldsplanlægning og indsamlingsordninger i Danmark. Ifølge Kommunernes Landsforening skal hver kommune udarbejde lokale affaldsplaner og regulativer, der fastlægger indsamlingsordninger for de enkelte affaldstyper. Dermed er kommunerne også aktører i forhold til læringsspillet, da de vil bemærke en forøgelse i korrekt sorteret affald, hvilket kan forbedre genanvendelsen af affaldet mindske yderligere manuel arbejdskraft ved genanvendelsesstationer.

1.4.3 Tertiære aktører

Affaldsselskaber og genbrugscentre

Affaldsselskaber er tertiære aktører, fordi de ikke direkte spiller med i læringsspillet, men de er nødvendige for at realisere sorteringen i praksis. De er afhængige af de indsamlede materialer og har interesse i, at spillet virker, da korrekt sortering fra borgerne øger mængden af affald, der kan genanvendes. Dermed hjælper spillets succes affaldsselskaberne.

1.5 Redegørelse for 'World Of Zuul'-koden

Som udgangspunkt for programmet er der blevet udleveret et framework for et tekstbaseret spil, med formålet at bygge videre på dette til udviklingen af spillet. Frameworket er bygget op omkring nogle simple funktionaliteter, der gør det nemt at udvikle ud fra.

Der dannes grundlag for et levelsystem, der er bygget op omkring "Nodes" og "Spaces", der samles i en verden. Et Space er et "sted" eller "rum", som bygger ovenpå Node. En Node indeholder den basisinformation, der er nødvendig for en lokation: Navn og relation. En relation eller "Edge" er frameworkets definition for en relativ "vej" fra en Node til en anden. På denne måde er det muligt at definere, hvilke Noder der leder til hinanden.

Disse Noder, forbundet via Edges, bliver lagt ind i en verden, som kan håndtere flere Noder, som hver især kan håndtere flere Edges. Med denne opbygning er det muligt, ud fra det meget simple framework, at opbygge en verden af relationer til forskellige rum. Dette giver samtidig mulighed for at bygge ovenpå disse Noder ved at udvide definitionen af, hvad et Space er, eller tilføje nye klasser, der udvider Node-klassen.

Yderligere er frameworket bygget med et kommandosystem, der giver en basal kommandobaseret interaktion med disse Noder i verdenen. Systemet er blokerende, da det er styret af brugerinput. Når en kommando sendes, udføres handlingen tilknyttet den givne kommando, hvorefter der ventes på næste input. Der er desuden stadig mulighed for at udvide systemet og tilføje yderligere funktionalitet.

1.6 Problemformulering

På baggrund af tidligere analyse af problemet, blev følgende problemformulering dannet:

"Kan vi øge viden om korrekt affaldssortering hos unge fra 10-19 år i Danmark og dermed mindske forkert affaldssortering og øge genanvendelsen af vigtige ressourcer, gennem et læringsspil med fokus på trial & error-princippet?"

Underspørgsmål

- Hvad bliver gjort lige nu for at øge viden om affaldssortering?
- Hvad er et læringsspil?
- Hvordan udvikles et læringsspil?
- Hvad er affaldssortering?
- Hvordan sorterer man affaldet ansvarligt?
- Hvad er konsekvenser af forkert eller uansvarlig affaldssortering?
- Hvordan kan man øge viden gennem et læringsspil?
- Hvordan kan man appellere til og motivere målgruppen?
- Hvem kunne drage nytte af øget viden (dvs. målgruppen)?

1.7 Metoder

I dette projekt har vi anvendt en kombination af udviklingsmetoder og læringsprincipper for at udvikle et læringsspil med fokus på korrekt affaldssortering. Projektet er blevet udført som en iterativ udviklingsproces, hvor vi løbende og gradvist udbygger og forbedrer projektet.

Vi har taget udgangspunkt i ingeniørmodellen gennem vores udviklingsproces. Her var fokus på at udvikle et virkende spil først, og dernæst ville vi videreudvikle gennem #2 iteration, hvor vi også havde mulighed for at forbedre vores struktur og brugeroplevelsen.

I analyse- og designfasen er der blevet brugt objektorienterede metoder, herunder noun/verb-analyse, CRC-kort og UML-klassediagrammer, til at identificere systemets klasser, ansvar og relationer. Disse metoder har dannet grundlaget for vores kodebase, og sørget for, at den var struktureret og sammenhængende.

Trial & error-metoden bygger på, at læring sker gennem gentagne forsøg, hvor handlinger afprøves, og deres konsekvenser observeres. Metoden er en naturlig del af den måde, mennesker lærer på, og anvendes både i hverdagen og i udviklingen af ny teknologi. Fejl betragtes ikke som noget negativt, men som en vigtig del af læreprocessen, da de viser, hvilke løsninger der ikke fungerer, og dermed bringer brugeren tættere på det ønskede resultat[15].

Denne tilgang hænger tæt sammen med brugen af læringsspil, som er en pædagogisk metode, der gør læring mere aktiv og motiverende. I læringsspil opnår brugeren viden ved at udforske spillet, afprøve forskellige handlinger og modtage feedback på sine valg. Gennem point, feedback og stigende udfordringer understøttes en gradvis opbygning af forståelse for både teori og praksis[16].

Ved udviklingen af spillet tages der derfor udgangspunkt i trial & error-princippet, hvor brugeren lærer gennem sine handlinger og de konsekvenser, disse medfører. Korrekt sortering belønnes med point, mens forkert sortering resulterer i feedback i form af infotekst samt en straf i spillet. Denne metode er valgt for at understøtte aktiv læring, særligt hos projektets målgruppe af unge i alderen 10–19 år[15].

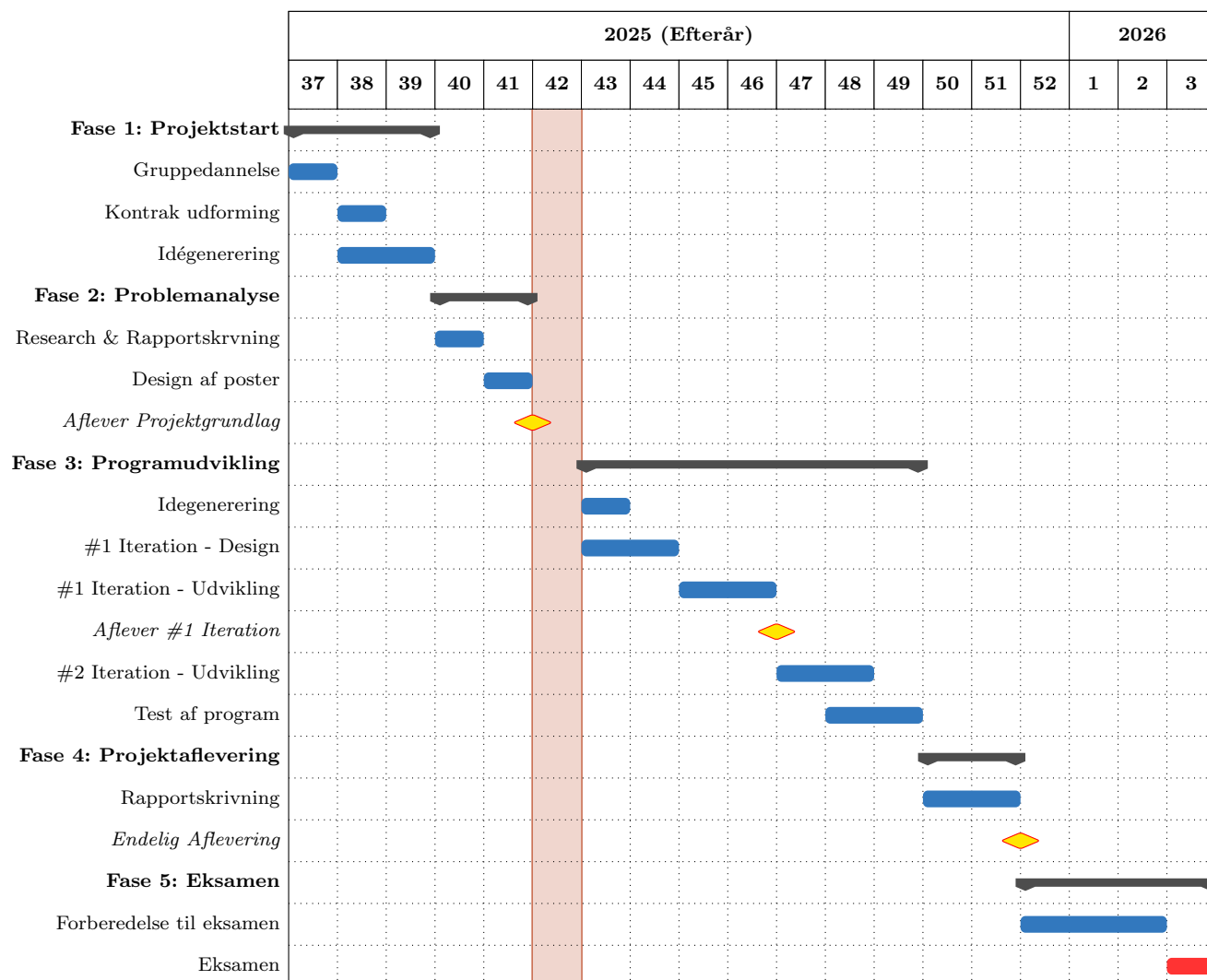
Derudover har vi også anvendt testing som en metode til kvalitetssikring af løsningen. Dette er blevet implementeret gennem løbende manuel test af spillet og simple unit tests. Her var målet at verificere, at programmets funktionalitet lever op til de krav, vi har opstillet. Resultaterne er anvendt til at identificere fejl og forbedringsmuligheder.

Samlet set har de valgte metoder sikret, at vi har haft en struktureret udviklingsproces, hvor vi har inddraget både tekniske og læringsmæssige aspekter til at besvare projektets problemformulering.

1.8 Tidsplan

Vores tidsplan er illustreret nedenfor, på Figur 2, hvor vi har udarbejdet en overordnet plan for projektets forløb. Vi har bygget den som en Gantt-diagram. Tidsplanen viser de forskellige faser, som projektet er opdelt i, samt de tilhørende aktiviteter og opgaver. Deadlines er markeret som milepæle,

ferieperioder er fremhævet med orange felter, og eksamensperioden er angivet i rødt i den sidste uge. Tidsplanen giver et samlet overblik over projektets progression, fra projektstart til eksamen.



Figur 2: Tidsplan for projektet.

2 Krav

2.1 #1 Iteration

2.1.1 Spilbeskrivelse

Spilbeskrivelsen er vigtig for at forklare brugeren, hvad vedkommende kommer til at opleve, og hvad de skal forvente af programmet. I et projekt med mange programmører og perspektiver, der adskiller sig fra hinanden, vil alle have et unikt syn på problemet. Derfor har vi i dette projekt valgt at lave en sammensætning af alles beskrivelser. Først skrev alle to beskrivelser til vores spilkoncept. Derefter sammenlignede vi dem og sammensatte dem til én overordnet spilbeskrivelse, der inkorporerede elementer fra alle forslag.

Sammensætning af vores beskrivelser

”Bliv en skraldehelt! Er du klar til at redde verden, én bananskræl ad gangen? I dette spil falder affald ned fra toppen, som i Tetris. Din mission er at sortere det korrekt i de rigtige skraldespande, før tiden løber ud. Lær af dine fejl, optjen point, og se, om du kan slå dine

venners highscore! Hver rigtig sortering er en sejr for planeten. Klar til at spille bæredygtigt og vise, hvem der er byens sejeste skraldehelt?”

2.1.2 Noun/Verb analyse

Til dette projekt har vi gennemført en noun/verb-analyse. Vi gjorde dette for at skabe et klart overblik over, hvilke dele programmet består af, og hvordan disse dele hænger sammen. Analysen blev brugt til at identificere objekter (klasser) og handlinger (metoder) ud fra beskrivelser af spillets funktioner.

Først startede vi med at gennemgå idéen og formulere en programbeskrivelse - en detaljeret beskrivelse af hvordan programmet virker og hvad det skal kunne (set i udsnittet nedenfor). Denne programbeskrivelse kan vi dermed bruge til at identificere navneord og udsagnsord i analysen. Derudover benytter vi også programbeskrivelsen i andre aspekter af udviklingen, som f.eks. ved CRC-kort eller kravspecifikationen.

Udsnit af programbeskrivelsen for menu-strukturen. Den fulde programbeskrivelse kan findes i bilagene.

Når spillet åbnes, skal menu-skærmen vises. Menu-skærmen skal indeholde 4 knapper:

- **PLAY:** Fjerner menu-skærmen og starter spilcyklus.
- **OVERSIGT:** Fjerner menuen og starter oversigt-siden.
- **CREDITS:** Fjerner menuen og starter credits-siden.
- **AFSLUT:** Lukker spillet.

Interaktion: “PLAY” knappen er markeret til at starte med. Spilleren kan skifte markering med op/ned piletasterne og vælge en knap med “enter”.

Oversigt-siden:

Indeholder en liste med skraldespande, en til hver af de fem skraldespande. Når man markerer en af skraldespandene, understreges den individuelle skraldespands information.

Skraldespand informationsside:

Her kan spilleren se, hvilket skrald der hører til i den valgte skraldespand.

Credits-siden:

Her kan spilleren se en liste over udviklere og deres roller.

...

En komplet programbeskrivelse og noun/verb analyse kan findes i bilagene.

Analysen giver et bedre grundlag for at strukturere den efterfølgende programmering. Den hjælper med at oversætte kravene til objektorienteret programmering, hvor det bliver tydeligt, hvilke klasser programmet skal bestå af, og hvilke metoder, der skal indgå i hver klasse.

Analysen sikrer derfor, at programmet bliver opbygget på en konsistent og velstruktureret måde, og at alle i gruppen har en fælles forståelse af systemets struktur og funktionalitet. Derudover sparer dette trin også tid senere i processen - f.eks. ved at identificere en klasse som ”skraldespand” allerede nu, undgår vi at dette bliver oprettet som en simpel variabel. Det gør dermed, at vi kan opbygge klassen tidligere i processen og spare tid på at omstrukturere senere.

Efter at navneord og udsagnsord var identificeret, blev de struktureret i en ufiltreret liste, som kan ses i Tabel 2

Verb	Noun
Åbne	Spil
Vise	Menu
Indeholde	Skærm
Fjerne	Spilcyklus
Start	Knap
...	...

Tabel 2: Udsnit af den indledende Noun/Verb analyse før filtrering.

På den måde har vi brugt noun/verb-analysen til at udlede de klasser og metoder, som danner grundlaget for spillets struktur. For eksempel blev ord som spil, menu, skraldespand og infotekst identificeret som klasser, mens ord som åbne, vise, starte, tjekke og gemme blev identificeret som metoder. Resultatet af analysen gjorde det muligt at udarbejde en overskuelig klasse- og metodeoversigt, som senere bliver brugt til at designe spillets kode og spilloop (se Tabel 3).

Verb (Potentielle metoder)	Noun (Potentielle klasser)
Åbne	Spil
Vise	Menu
Fjerne	Knap
Start	Skraldespand
...	...

Tabel 3: Udsnit af den filtrerede liste over metoder og klasser.

2.1.3 CRC-Kort

Før implementeringen af programmet udarbejdede vi CRC-kort som led i den objektorienterede analyse. Formålet var at identificere de centrale klasser i programmet samt definere deres ansvar og samarbejdsrelationer. Ved at opstille klasserne i programmet på forhånd opnåede vi et mere struktureret overblik over systemets arkitektur.

I processen for at formulere CRC-kortene begyndte gruppen først at gennemgå de tidligere definerede klasser og metoder fra noun/verb-analysen. Herefter blev en klasse udvalgt og udfyldt med det ansvar, vi mente, klassen skulle have. Nedenfor på Figur 3 ses opbygningen af Game-klassens CRC-kort. Det stod meget tydeligt for gruppen, at Game-klassen ville være en central klasse i programmet og dermed ville indeholde væsentlig funktionalitet. Game-klassen fik f.eks. ansvaret for at køre et "gameloop", der skulle tage imod input fra spilleren, styre spillets pausestadie, mv.

Game	
Ansvar	Samarbejde
■ Holder styr på game state ■ Holder styr på score og liv ■ Styrer visning på skærmen ■ Styrer hvilket skrald der falder ■ Kører "gameloop": - Input fra spiller - Pause/Afslut - Øge sværhedsgrad	- TrashObject - Screen - Trashcan - PlayerScore - InfoText

Figur 3: CRC-kort for Game.

På samme måde gennemgik gruppen alle klasserne og opsatte ansvaret for hver enkelt klasse. De prædefinerede klasser blev ikke kun oprettet, men også tilpasset. F.eks. blev der i noun/verb-analysen identificeret klassen "Leaderboard", som i løbet af denne proces blev ændret til PlayerScore-klassen. Både navn og funktionalitet blev her ændret, i takt med at gruppen planlagde og idégenerede på programmet. Dette var god ændring at foretage nu, frem for senere i processen.

Efter at alle kort var defineret, stod arbejdet på at skabe en generel ide om samarbejdet mellem klasserne. Her blev ansvaret for hver af klasserne gennemgået, og det blev vurderet, hvilke andre klasser det var relevant at kende til, for at virke optimalt. Nedenfor på Figur 4 og ovenfor på Figur 3 kan det observeres, at selvom Game-klassen skal kende til InfoText-klassen, behøver InfoText-klassen ikke nødvendigvis at kende til Game-klassen.

InfoText	
Ansvar	Samarbejde
■ Skal kunne registrere/kontrollere interaktion (at det bliver læst) ■ Skal kunne åbnes og lukkes ■ Skal kunne vise noget tekst	- Trash

Figur 4: CRC-kort for InfoText.

En komplet oversigt over CRC-kort kan findes i bilagene.

2.1.4 Kravformulering

Som udgangspunkt fik gruppen udleveret nogle krav til opgaven, herunder:

- Den udleverede kildekode skal udforskes som en del af problemanalysen.
- Den udleverede kildekode skal udvides i løbet af gennemførselsfasen.
- Alle medlemmer af gruppen skal have adgang til alle versioner af koden.

Disse krav lagde fundamentet for kravspecifikationerne. Derudover blev der defineret yderligere krav til systemet, hvor der tages udgangspunkt i programbeskrivelsen samt de mål, vi havde til spillet. Dermed kunne vi opdele systemets krav i to kategorier: funktionelle og non-funktionelle krav.[17]

For at identificere kravene har vi systematisk gennemgået vores programbeskrivelse og stillet spørgsmålene:

- ”Hvad skal programmet kunne” - For at finde de funktionelle krav
- ”Hvordan skal programmet udføre disse funktioner” - For at finde de non-funktionelle krav

Tager man udgangspunkt i første del af vores programbeskrivelse, og stiller de ovennævnte spørgsmål, indser man at programmet skal kunne vise nogle forskellige skærme, og at det skal håndtere brugerens input. Disse ting skriver vi så ind som funktionelle krav, og det blev til krav F1.01 og F1.09. Denne metode brugte vi gennem hele vores programbeskrivelse, for først at definere, hvad vores program skal.

Id	Navn	Beskrivelse	Status
F1.01	Vis menu-skærme	Programmet skal kunne vise flere forskellige menu-skærme	Færdiggjort
F1.02	Vis tekst & infobokse	Programmet skal kunne vise tekst og forskellige info-bokse.	Færdiggjort
F1.03	Gameloop	Programmet skal have spil-loop, så spilleren kan spille uendeligt. Ydermere skal spillet gradvist blive sværere. (hurtigere).	Færdiggjort
F1.04	Start & afslut spil	Programmet skal kunne startes og afsluttes.	Færdiggjort
F1.05	Objekt håndtering	Programmet skal kunne håndtere objekter, herunder: spawn, falde, kontrollere, osv.	Færdiggjort
...	...	...	...

Tabel 4: Udsnit af funktionelle krav fra #1 Iteration. Den fulde liste findes i bilag.

Udover at gå igennem spilbeskrivelsen, identificerede vi flere krav gennem brainstorming og analyse af målgruppen. Som et eksempel til de non-funktionelle krav, fokuserede vi bl.a. på brugeroplevelsen, og et krav var, at programmet skal kunne køres på enhver Windows-computer, og at det skal være nemt at bruge, som så blev til krav NF1.01 og NF1.02.

Id	Navn	Beskrivelse	Status
NF1.01	Kompatibilitet	Programmet skal kunne køres på en standard Windows computer.	-
NF1.02	Brugervenlighed	Programmet skal kunne være nemt at bruge for unge i folkeskole	Færdiggjort
NF1.03	Læringsorienteret	Spilleren af programmet skal kunne lære noget omkring affaldssortering ved brug af spillet i en længere, uspecificeret periode.	Færdiggjort (ikke testet)
NF1.04	Ydeevne	Programmet skal køre smoothly.	Færdiggjort
NF1.05	Læsbarhed	Teksten skal være læsbar, og størrelsen skal være passende.	Færdiggjort
...	...	...	...

Tabel 5: Udsnit af non-funktionelle krav fra #1 Iteration. Den fulde liste findes i bilag.

Efter første iteration havde vi implementeret de grundlæggende krav, der definerer spillets kernefunktionalitet. De identificerede krav dannede dermed den første kørende prototype, og gav os et solidt fundament for videre udvikling.

2.2 #2 Iteration

2.2.1 Krav udvidelser

#2 Iteration byggede videre på funktionaliteten fra #1 iteration, og introducerede en række mere avancerede krav. Formålet var at gøre spiloplevelsen mere dynamisk og pædagogisk, og iteration 2 af vores kode resulterede dermed i et bedre spil og en bedre oplevelse.

F.eks. blev der tilføjet kravet F2.03, som ses i Tabel ??, for at skabe en bedre progression i spillet. Dette var blandt andet på baggrund af feedback fra midtvejsseminaret samt egne ønsker om udvidelse.

Id	Navn	Beskrivelse
F2.01	Fejlfri program	Ingen tydelig fejl og mangler, på kørende program. Her mener vi, at der ikke er noget der skal hindre brugerens spiloplevelse og/eller afvige dem fra spillets gang. (bugs)
F2.02	Lås af rum	De 5 forskellige rum (udover start rummet) skal låses indtil alle typer skrald fra det tidligere rum er fundet.
F2.03	Tilføj skrald	Forskellige typer skrald skal indsættes i hvert rum, så spilleren har mulighed for at opdage nye slags skrald fra rum til rum.
F2.04	Level selector skal vise opdaget skrald	Level selectoren skal vise hvilke slags skrald er opdaget for hvert rum.
F2.05	Gem og hent opdaget skrald og forskellige highscore	Spillet skal kunne gemme og hente forskellige highscores for hvert rum, samt gemme hvilke slags skrald er opdaget for hvert rum.
...	...	...

Tabel 6: Udsnit af nye krav til #2 Iteration. Den fulde liste findes i bilag.

2.3 Brug af krav

Kravene gav et klart billede af, hvad spillet skulle kunne, og hvilke kvaliteter det skulle have. Dette skabte en generel rute og guide til vores udviklingsproces, hvor vi undervejs kunne markere de krav, som var opfyldt. Dette gjorde det lettere at holde styr på vores fremgang og se, hvor langt vi var i forhold til vores mål.

Derudover brugte vi kravene til at sikre, at løsningen opnåede den rigtige kvalitet, da vi kunne sammenligne en funktion med de relevante krav for at se, om den faktisk levede op til det, vi havde defineret i starten. Dermed kunne vi løbende rette fejl og forbedre dele af spillet, før vi gik videre i processen.

3 Design

3.1 #1 Iteration

Fokus for #1 Iteration var at skabe et design, der lægger grundlaget for spillet. Formålet var at få skabt et MVP (Minimum Viable Product), således at programmet opfylder projektets og vores krav. Derudover var det også vigtigt at holde os til kravene, da #2 Iterations formål er at bygge videre på spillet.

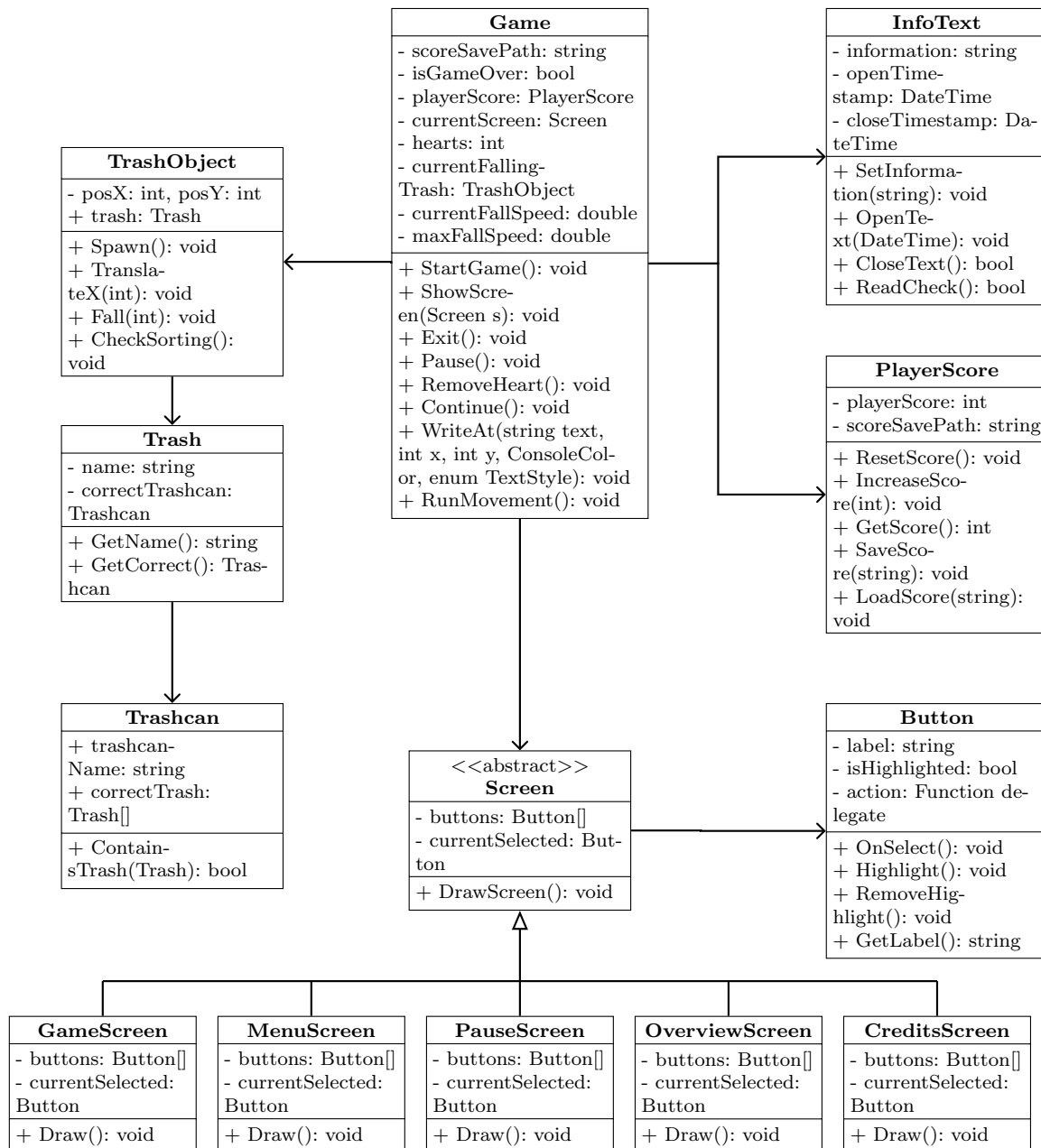
Under første iteration blev der også udarbejdet et overordnet level-design, bestående af seks levels, der er ens, men hver især har forskellige faldhastigheder af skraldet.

3.2 UML-klassediagram

Vores første trin var at udforme et UML-klassediagram, for at skabe et fælles overblik over, hvordan programmet og dets arkitektur skal opstilles, allerede før vi begynder med at kode programmet. Dette

gør det nemmere at identificere (og håndtere) problemer og krav tidligt i processen, og sørgede for, at vi vidste præcis, hvordan kodestrukturen skal opstilles.

Figur 5 viser, hvordan vores indledende diagram var opsat. Baseret på de klasser, der tidligere blev identificeret i vores CRC-kort, skabte vi kasser for hver af klasserne, hvorefter vi udfyldte klasserne med deres attributter og metoder.



Figur 5: UML-Klassediagram over systemet. #1 Iteration

Hvor vi i CRC-kortene beskrev ansvar overordnet med tekst, oversatte vi i denne proces ansvaret til konkrete metoder og attributter. Som eksempel, ses klassen "Game" på Figur 5, hvis ansvar bl.a. inkluderede at "holde styr på score og liv".

Under Game-klassens attributter beskrives så en variabel til en instans af en PlayerScore, samt en integer ved navn hearts. På denne måde får vi konverteret den konceptuelle beskrivelse af denne del af klassens ansvar, om til konkrete attributter. Eftersom scoren er mere kompleks og har behov for

load, save og andre funktioner, benyttes der her PlayerScore-klassen i modsætning til livene, der har simple behov og dermed kan repræsenteres alene af en integer.

Ligeledes formuleres metoder som RemoveHeart. En metode med hensigten at håndtere funktionaliteten ved tab af et hjerte. Yderligere bliver der indtegnet relationer mellem klasserne. Igen med fokus på Game-klassens formål om at håndtere scoren, kan det ses, at Game-klassen har en direkte relation til PlayerScore-klassen. Dette gør det muligt at skabe dette overblik over programmets funktionalitet.

Nu, baseret på dette, ved vi, at Game-klassen skal kunne håndtere spillerens score og opbevare den i en instans af PlayerScore-klassen. Dette opfylder også ansvaret "Opbevarer spillerens score" for PlayerScore-klassen. På denne måde bliver en potentielt tvetydig beskrivelse af ansvar, som "at holde styr på score", oversat til en konkret struktur, der er nemmere at overskue.

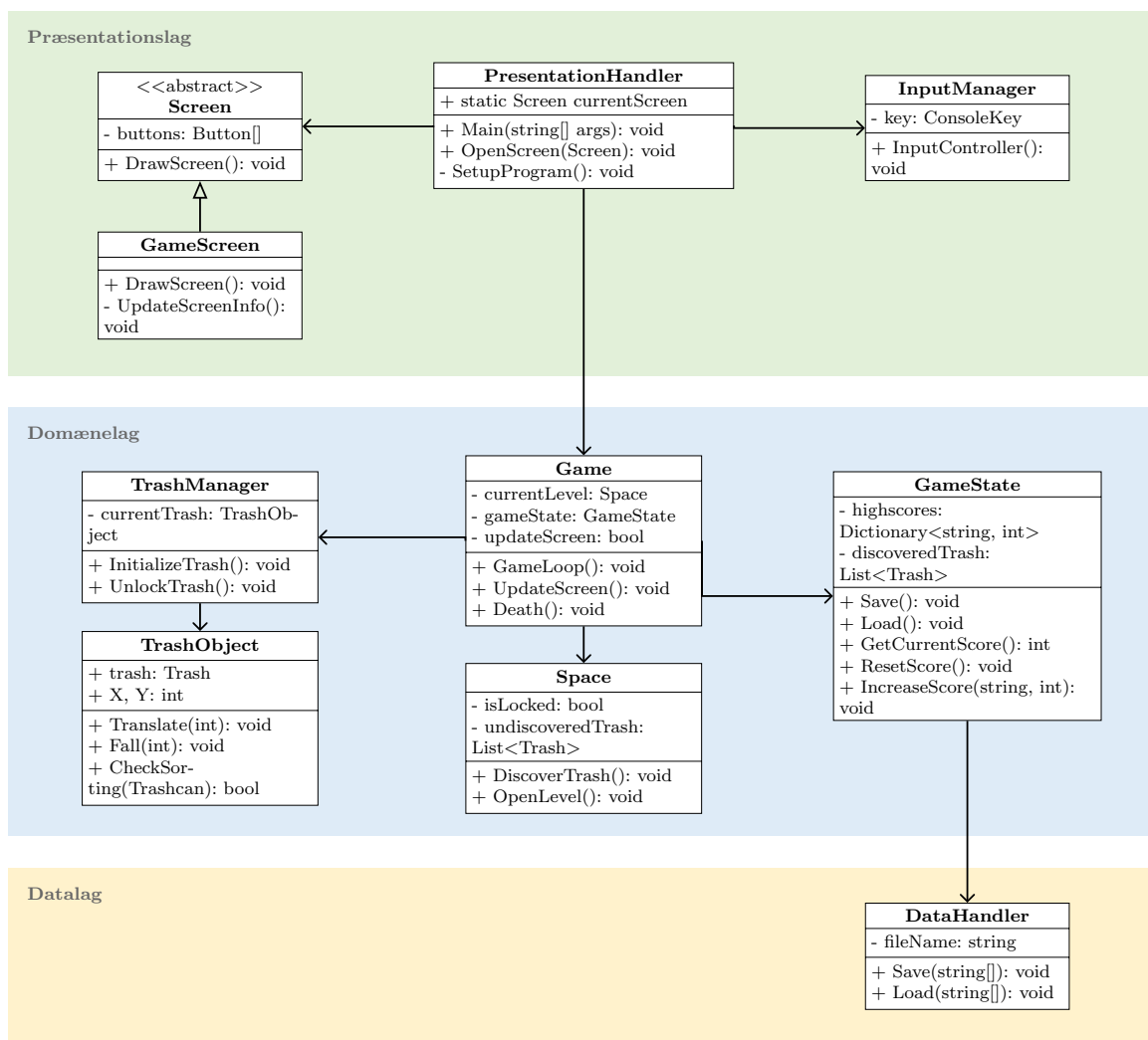
Samme fremgangsmåde er benyttet til iterativt at udforme resten af diagrammet. Ved at gennemgå og få specificeret ansvaret og forholdene mellem klasserne, fremgår en klar struktur for programmet.

Game-klassen fungerer som controller, der styrer spillets logik og delegerer opgaver til relevante klasser. F.eks. holder den styr på den nuværende skærm med attributten currentScreen af Screen-klassen. Den benytter sig af den nedarvningsstruktur specificeret ved en overordnet skærm-klasse og nogle subklasser, der udformes og specialiseres baseret på deres enkelte ansvar.

Ydermere skabes der kendskab fra Game-klassen til TrashObject-klassen, som er det objekt, der faktisk bruges til at kunne flytte på og sortere skrald. Ligeledes etableres en relation til Trash-klassen, som igen kender Trashcan-klassen. Denne struktur viser, hvordan det bagvedliggende skraldesystem opbygges og bruges af Game-klassen.

3.3 #2 Iteration

Det fremgår tydeligt af den tidligere iteration, at softwarearkitektur ikke været et fokus. Derfor har en af målsætningerne med #2 Iteration været at restrukturere koden for at få en klar lagopdeling. I den forbindelse blev der udformet et mere overordnet UML-klassediagram, for at visualisere den nye opdelte struktur, samt en oversigt over ændringer i koden.



Figur 6: UML-klassediagram over udvalgte dele af systemet. #2 Iteration

På Figur 6 ses et UML-klassediagram med de vigtigste elementer der illustrerer udviklingen fra #1 til #2 Iteration. Ydermere visualiserer diagrammet de enkelte klassers placering i lagene i softwarearkitekturen, som blev indført som del af #2 Iteration.

En af de største ændringer fra #1 Iteration var GameState, som fungerer som den nye og forbedrede PlayerScore. GameState har samme funktionalitet som tidligere, men bliver nu udstyret med funktionaliteten til at holde styr på highscores og opdaget skrald for alle levels. Årsagen til denne ændring kommer fra kravet F2.05, som siger, at programmet skal kunne hente og gemme highscores og opdaget skrald for de enkelte levels. Ydermere er Save- og Load-funktionaliteten blevet ekstraheret til datalaget, hvor det nu er Datahandler-klassen, der står for al kommunikation med filsystemet.

Denne DataHandler-klasse var nødvendig at tilføje, eftersom det i denne iteration var indført, at de enkelte levels hver især indeholder forskelligt skrald, der skulle ”opdages” ved at spille spillet og få en highscore høj nok til at låse op for det.

TrashManager-klassen er også en ny tilføjelse i #2 Iteration, som nu står for at holde styr på og kontrollere det TrashObject, der benyttes i spillet. Dette er gjort for at få en bedre opdeling af koden, samt sørge for, at Game-klassen, som tidligere i #1 Iteration stod for en stor del af logikken, bliver delt op.

Til sidst er en væsentlig ændring, at kodens ”Entry point” eller Main-metode er flyttet til en ny PresentationHandler-klasse. Denne klasses formål er at tage noget af det tidligere ansvar fra Game-

klassen, som at styre skærmen, køre main-loopet osv., så Game-klassen igen bliver mere specialiseret med et konkret formål. Derudover er Main-metoden også flyttet for bedre at understøtte den nyindførte lagdeling.

3.4 Design beskrivelse

3.4.1 Softwarearkitektur

Som udviklingsprocessen tog form, blev det åbenlyst, at det indledende design defineret i #1 Iteration ikke var holdbart ift. de behov, der var for #2 Iteration. Kodebasen var begyndt at blive større end først forventet og mere kompleks. Dette gjorde det svært at finde rundt i koden og endnu sværere at lave ændringer. Derfor blev det besluttet, at programmet skulle gennemgå en refaktoring, hvor kodebasen gennemgås og forbedres. En sådan forbedring var at omstrukturere alle klasser og funktionaliteter, så de passer ind i en lagdelt softwarearkitektur. Denne arkitektur ville være tredelt, således at:

- Præsentationslaget - står for interaktionen med spilleren, altså input og det visuelle (UI).
- Domænelaget - står for den bagvedliggende logik og funktionalitet, altså at få spillet til at køre.
- Datalaget - står for datahåndtering, altså at kunne hente og gemme spildata.

Med denne klare opdeling, kombineret med vores UML-diagram, fik vi først og fremmest en bedre fornemmelse af kodens struktur, som gjorde det nemmere at implementere og vedligeholde koden mht. krav NF1.06. Men det skabte også grundlag for implementeringen af flere af de ændringer, der gjorde sig gældende for #2 Iteration.

Domænelaget & World of Zuul

Dette lags formål er at styre programmets bagvedliggende tilstand og logik. Centralt til dette er Game-klassen, som står for håndtering af gameLoop-logikken, der styrer spillet. Det er dette loop, hvor den nuværende instans af TrashObject (det stykke skrald på skærmen i spillet) bliver opdateret i position og sørger for at tjekke, om den har ramt en skraldespand.

Ydermere er en egenskab af Game-klassen også, at den holder styr på nogle af de centrale informationer for domænelaget. Så som: Skal/bliver der vises/vist infotekst? Skal skærmen opdateres? mv.

I samarbejde med Game-klassen er TrashManager-klassen central for afviklingen af spillet, da det er den, der står for at håndtere og spawn det TrashObject-instans, der bruges i spillet. Ydermere står TrashManageren også for at låse op for nyt skrald løbende, som spillet afvikles.

Disse klasser som Game og TrashManager er eksempler på, hvad domænelagets opgave er. Der tegnes ikke nødvendigvis skrald på skærmen - det ved domænelaget ikke noget om, men positionen opdateres internt, således at præsentationslaget ved, hvor det skal tegnes.

Det er også på dette lag, at der er taget funktionalitet fra World of Zuul-koden. Her er det bagvedliggende level-system bygget op med udgangspunkt i den udleverede kode. Her er klasser som Node, Space og World blevet genbrugt. Arvstrukturen af klasserne er bevaret, men indholdet og ansvaret er justeret til bedre at passe spillet.

Konceptet af et "rum" blev modificeret til udelukkende at være noget bagvedliggende logik, der ikke gøres kendt for spilleren. Grundstrukturen er stadig den samme, hvor Space nedarver fra Node og World sætter en "verden" af Spaces op, men et "rum" indeholder nu bare informationer om det pågældende level. Derfor kan vi bruge currentLevel-variablen fra Game-klassen til at justere kendt skrald, sværhedsgrad og highscore for hvert level.

Datalaget & Persistens

I forbindelse med implementering af kendt skrald og highscore, blev der i #2 Iteration defineret kravet F2.05, som siger, at spillet skal kunne gemme og hente spildata. Derfor er datalaget etableret for udelukkende at håndtere persistens og kender dermed ikke til de andre lag.

Laget består af klassen DataHandler, der arbejder sammen med GameState-klassen fra domænelaget. GameState interagerer med DataHandler, mens DataHandler interagerer med computerens filesystem. Her bliver data gemt i CSV-filformat, da filformatet stemmer overens med kravet om at kunne gemme.

Præsentationslaget

Med den underliggende logik og data etableret, er det også nødvendigt at have en måde at præsentere spillet for brugeren. Formålet med dette lag er at håndtere alt det visuelle UI samt interaktioner med spilleren. Her bliver spillerens input bl.a. håndteret. Her er klasserne Button og InputManager relevante, som sammen lader spilleren navigere rundt i menuen og spillet med keyboard-input.

Her kommer Screen-klassen ind, som er den overordnede klasse, som alle de skærme, der er defineret i spillet, stammer fra. Den indeholder f.eks. funktionalitet til at behandle de inputs, der håndteres af InputManageren, således at de kan repræsenteres visuelt i konsollen.

På den måde har programmet mulighed for at modtage og behandle inputs samt vise dem frem til spilleren. Men for at forbinde disse elementer sammen, er klassen PresentationHandler defineret. Denne klasse er programmets ”indgang”, da den indeholder Main-metoden. Den står for at forbinde inputs med handlinger, samt kommunikationen med domænelaget.

Hjælpekomponenter

Endnu en tilføjelse i #2 Iteration var nogle hjælpefunktioner, hvis formål er at skabe en bedre centraliseret struktur i projektet samt mindske kodegentagelser. Dette inkluderer klasser som Theme og ConsoleDrawer på præsentationslaget, og Level på domænelaget.

Theme- og Level-klassernes primære formål er at centralisere farver og levelnavne på ét sted. På den måde skal ændringer kun foretages ét centralt sted, hvis programmet senere udvides.

ConsoleDrawer har et lignende formål. Forskellen er dog her, at ConsoleDrawer er en samling af gængse metoder, der benyttes på tværs af præsentationslaget. Hvis det senere var nødvendigt at ændre koden for, hvordan en kasse skal tegnes, vil ændringen her være universel for alle kasser, der tegnes. Dette er fordelagtigt i projekter som dette, hvor det giver mening at gå tilbage og ændre funktionalitet.

Arkitekturrefleksion

Tidligere nævnes det, at hele konceptet om lagdeling af softwarearkitekturen først introduceres i #2 Iteration af processen. Dette skyldes opgavens forløb, hvor viden og koncepter løbende introduceres sideløbende med udviklingen af projektet. Dette har medført, at det i processen ofte har været relevant at gå tilbage og lave ændringer i tidligere implementeret funktionalitet.

Dette gøres specielt tydeligt ved implementeringen af den lagdelte arkitektur. Programmet har nogle mangler, f.eks. en mangel på et ”interface- et mellemlid, der styrer og begrænser adgangen til underliggende lag. Lige nu tilgår præsentationslaget f.eks. domænelaget adskillige steder. Et interface ville være med til at øge effektiviteten af lagdelingen, da det kun er ét sted noget skal redigeres, hvis håndteringen ændres.

3.4.2 Brugergrænseflade

Designet har ikke gennemgået nogle større ændringer fra de indledende valg i #1 Iteration - kun mindre rettelser, layout og beskrivelser.

Valgene taget under udformningen af brugergrænsefladen er tæt forbundet med de non-funktionelle krav stillet til opgaven, da de vedrører ting som brugervenlighed, tydelighed osv.

Designfilosofi & Begrænsninger

Når der arbejdes med et spil i konsolterminalen, er der en del grundlæggende begrænsninger, som der skal arbejdes udenom. En af de største af disse er de begrænsede tilgængelige redskaber til at kunne udtrykke noget visuelt. Essentielt er der kun ASCII-karakterer, Unicode-symboler, emojis og farver. Ingen mulighed for billeder eller grafik - en udfordring, når der skal udvikles et spil. Dette gælder især, når der skal udvikles et spil, der skal appellere til den yngre målgruppe af børn i dag.

Yderligere er en begrænsning mulighederne for spillerinput. Input i konsollen er som udgangspunkt begrænset til keyboard-inputs, hvilket eliminerer muligheden for at benytte musen til interaktion eller navigation.

En anden begrænsning af konsollen er dens "grid". Konsollen består af linjer og kolonner, hvor der kan stå ingenting eller et symbol. Aspektforholdet (højde vs. bredde) af disse er ikke kvadratisk, hvilket gør det svært at tegne geometriske former.

På baggrund af dette er der opstillet kravene NF1.02, NF1.05, NF1.08 og NF1.10, der alle er relevante for designet og brugergrænsefladen i spillet. Så udfordringen i dette projekt har været at skabe et brugervenligt og interaktivt spil, der er klart og tydeligt samt nemt at læse og navigere.

Opbygning af UI

Trods disse begrænsede værktøjer har det dog været muligt at opstille et UI, der opfylder kravene. Designet af menusiderne i spillet har været stort set urørt i #2 Iteration, eftersom ingen større ændringer blev lavet til spillets indhold.

På tværs af næsten alle skærme er der benyttet et tema. Siderne er bygget op med en ASCII-kunst titel i midten, øverst på skærmen (se Figur 7a). Der gøres brug af ASCII-kunst, fremfor normale ASCII-karakterer til titlen, for at skabe et større fokus og kontrast til den normale tekst. Titlerne benyttes på alle menusiderne på nær selve spilsiden, ses på Figur 8a og 8b.

Yderligere gøres der brug af diverse knapper og kasser i store dele af programmet. Dette kan f.eks. ses på Figur 7b. Her findes seks knapper - én for hvert level. Disse knapper, kasser og skærmens kant er opbygget af Unicode-symboler fra "Box Drawing-blokken (U+2500 - U+257F)". Dette er symboler, der er designet til at lave sammenhængende linjer. Generelt placeres knapper og andet på en måde, hvor de er ligeligt fordelte og har luft, som giver en bedre æstetisk oplevelse.

Levelselector-skærmen er opsat på en måde, der giver et nemt overblik over en spillers progression i spillet. Der vises tydeligt highscore og fundet skrald for hvert level. Bunden af skærmen er dedikeret til at fremvise det skrald, der er fundet, ligesom det er muligt at se, hvor mange stykker skrald der endnu ikke er opdaget. Yderligere er det indikeret med den gråtonede farve for nogle af knapperne, at disse levels er låst.

Denne opsætning og information, der bliver vist, er med til at give spilleren et nemt og overskueligt overblik, uden at give for meget information.

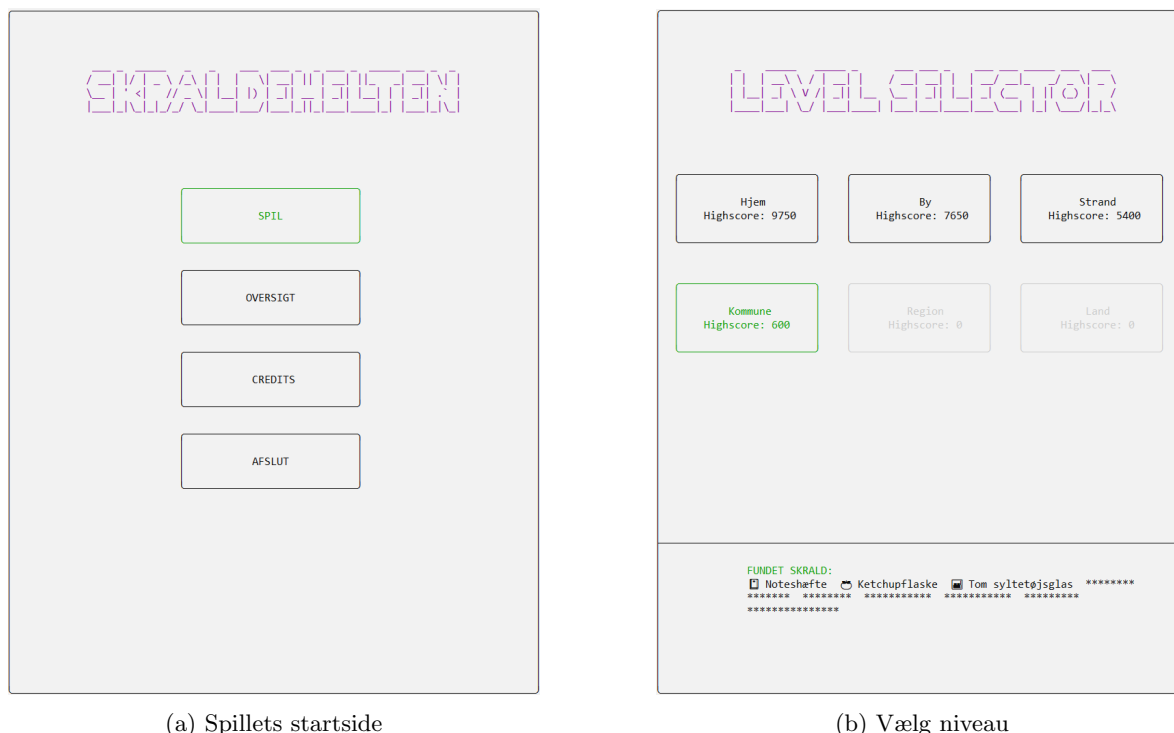
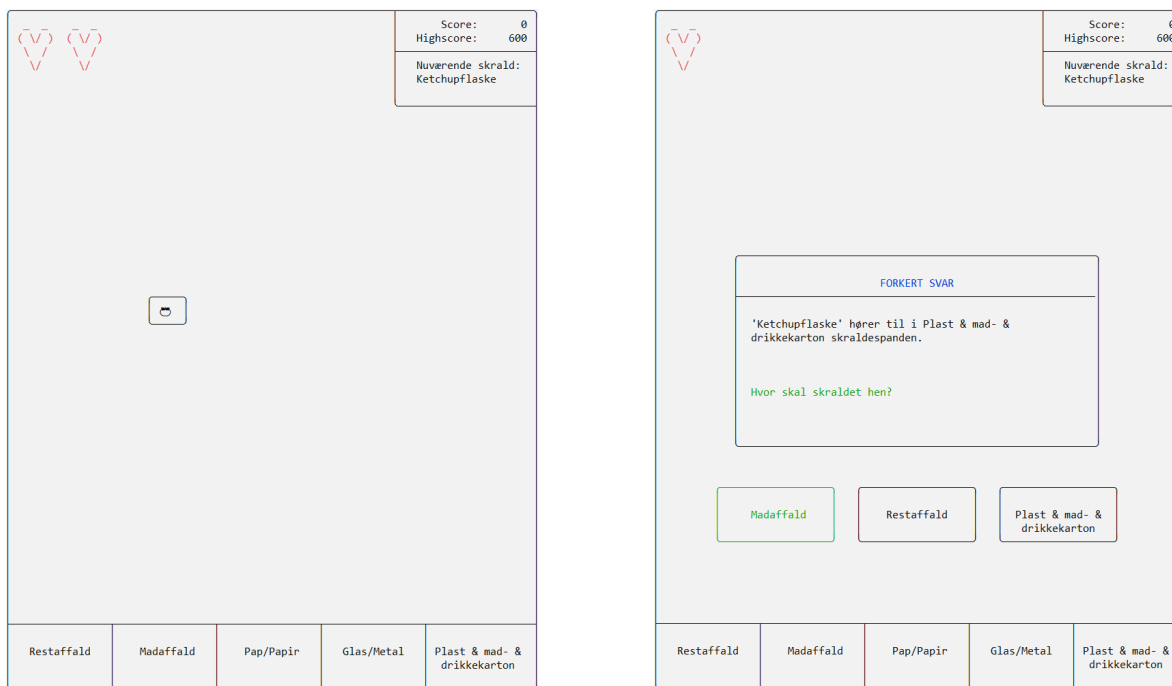


Figure 7: Her ses navigations-flowet i spillets menu før selve gameplayet starter.

Spilsiden (Figur 8a) gør ikke brug af en titel, for at fremme fokuset på selve spillet. Derfor er spilsiden også forholdsvis minimalistisk. Her er lagt særlig fokus på ligevægt. Der er fem centrale elementer af spillet: Skraldespande, liv, faldende skrald, score og skraldenavn.

Elementerne er fordelt således, at det giver luft til området, hvor skraldet falder, samtidig med at elementerne fordeles i en omvendt trekant. Livene er placeret i øverste venstre hjørne, scoren og skraldenavnet i øverste højre hjørne, og til sidst skraldespandene i bunden af skærmen. Denne fordeling af elementer er valgt frem for, f.eks. at placere alt informationen i en informationsbjælke i bunden af skærmen. Det er med til at centralisere spillerens syn og samle deres fokus på spillets vigtigste elementer samtidig med at have det faldende skrald i midten.

Dette designvalg modarbejdes dog delvist på baggrund af manglen på grafik. Da det ikke er muligt at visualisere det faldende skrald med et billede, er det valgt at repræsentere det med en emoji. Dette valg giver en visuel forskel på de forskellige stykker skrald, samt noget interessant at kigge på. Men det er ikke nok til at give en indikation af HVAD skraldet er. Derfor bliver dette skrevet øverst til højre, for at give spilleren en bedre idé om hvad de skal sortere. Dette resulterer dog i, at spillerens fokus glider op mod højre - væk fra spillet. Et forbedringspunkt, der kan berøres i videreudvikling af programmet. Eventuelt ved at indføre en GUI (Graphical User-Interface), frem for et CLI (Command-Line Interface).



(a) Spillet i gang

(b) Infotekst efter forkert sortering

Figur 8: Her ses gameloopet under selve gameplayet.

I løbet af #2 Iteration blev det gennem feedback og test blev det gjort klart, at menuserne ikke er specielt informative angående spillets formål. Det nævnes specifikt, at spillets formål først bliver klart, når selve spillet går i gang. Det er en feedback, der blev taget i betragtning til videreudvikling på et senere tidspunkt pga. tidsmæssige begrænsninger.

4 Implementering

4.1 #1 Iteration

I #1 iteration fokuserede vi på at etablere alle de grundlæggende funktioner, der skulle til for at skabe vores MVP. Dette omfattede blandt andet udvikling af menuer, spilskræmen, gameloopet, tastaturstyring, objekt-håndtering, korrekt sortering, livsystem og score. Nedenfor beskrives det, hvordan disse

krav blev implementeret i praksis, understøttet af kodeuddrag og forklaring.

4.1.1 Implementering af funktionelle krav

For at understøtte flere skærme udviklede vi et skærmsystem, hvor alle skærme arver fra en fælles superklasse:

```
3  Public abstract class Screen
4
5  public Button?[] buttons;
6  protected Button? currentSelectedButton;
7
8  public void MoveButtonSelection(int direction)
9  {
10     if (currentSelectedButton == null || buttons.Length == 0) return;
11     int currentIndex = Array.IndexOf(buttons, currentSelectedButton);
12     int newIndex = currentIndex + direction;
13     if (newIndex >= buttons.Length)
14     {
15         newIndex = 0;
16     }
17     else if (newIndex < 0)
18     {
19         newIndex = buttons.Length - 1;
20     }
21     currentSelectedButton = buttons[newIndex];
22     currentSelectedButton.Highlight();
23 }
24 public abstract void DrawScreen();
```

Listing 1: Udsnit af Screen.cs scriptet fra #1 iteration

Denne klasse definerer rammerne for alle skærme (menu, pause, oversigt osv.). Hver skærm har en DrawScreen-metode. Da det er en abstract metode, er der ingen prædefineret handling, så hver sub-klasse skal dermed implementere sin egen. Dette gør systemet fleksibelt og let at udvide.

Skærmskift håndteres centralt:

```
329 public static void OpenScreen(Screen screen)
330 {
331     Console.Clear();
332     currentScreen = screen;
333     DrawGameEdge();
334     screen.DrawScreen();
335 }
336 }
```

Listing 2: Udsnit af Game.cs scriptet fra #1 iteration

Når spillet skifter skærm, ryddes konsollen, rammen tegnes igen og den ønskede skærm tegnes. Dette opfylder kravet (F1.01) om at kunne vise flere menuer samt navigere mellem dem.

Ved forkert sortering vises en infoboks med en forklaring og et spørgsmål:

```

242 InfoText infoTextUniversal = new InfoText
243 (
244     "FORKERT SVAR"
245     "\"" + currentTrash.trash.GetName() + "\"" + "hrer til i " +
246     ↪ currentTrash.trash.GetCorrect().GetName() + "skraldespanden.",
247     "Hvor skal skraldet hen?"
248     currentTrash.trash,
249     trashcanSelected
250 );
251 infoTextShowing = true;
252 currentInfoText = infoTextUniversal;
253 infoTextUniversal.Show();

```

Listing 3: Udsnit af Game.cs scriptet fra #1 iteration

InfoText-klassen tegner en grafisk boks i konsollen og genererer multiple-choice svar. Dette giver spilleren øjeblikkelig læring og visuel feedback, hvilket opfylder kravet (F1.02) om tekst og infobokse.

Spillets hovedlogik ligger i gameloopet:

```

185 public static void GameLoop()
186 {
187     if (infoTextShowing)
188     {
189         return;
190     }
191     if (health >= 0)
192     {
193         Death();
194         return;
195     }
196     double fallInterval = 1 / currentLevel.GetFallSpeed() * 100;
197
198     // Initialize trash
199     if (currentTrash == null)
200     {
201         InitializeTrash();
202     }
203
204     ...
205
206     if (DateTime.Now - lastFallTime).TotalMilliseconds >= fallInterval)
207     {
208         currentTrash.Fall(1);
209         lastFallTime = DateTime.Now;
210     }

```

Listing 4: Udsnit af Game.cs scriptet fra #1 iteration

Gameloopet kører kontinuerligt og får skraldet til at falde i faste intervaller. Fall-hastigheden afhænger af niveauets fallSpeed, så spillet gradvist bliver sværere. Dermed opfyldes kravet (F1.03) om et uendeligt spil med stigende sværhedsgrad.

Hvert stykke skrald repræsenteres af et TrashObject:

```

25 public void Spawn()
26 {
27     positionX = 41;
28     positionY = START_Y;
29 }
30
31 ...
32
33 public void Translate(int xMove)
34 {
35     int step = 17;
36     int rowIndex = positionX / step;
37     int newX = rowIndex * step + step / 2 + xMove * step - rowIndex;
38
39     if (newX < 0) return;
40     if (newX > Game.gameWidth - 1) return;
41
42     positionX = newX;
43 }
44
45 /// <summary>
46 /// Får skraldet til at falde et givent antal pixels.
47 /// </summary>
48 public void Fall(int fallAmount)
49 {
50     positionY += fallAmount;
51     if (positionY >= Game.gameHeight)
52         positionY = Game.gameHeight;
53 }

```

Listing 5: Udsnit af TrashObject.cs scriptet fra #1 iteration

Her defineres skraldets opførsel:

- Spawn() placerer skraldet øverst på skærmen
- Translate() flytter det vandret
- Fall() får det til at falde nedad

Dette opfylder kravet (F1.05) om at kunne håndtere spilobjekter logisk og visuelt.

Koden der tjekker, om spilleren valgte den korrekte skraldespand:

```

77 public bool CheckSorting(Trashcan selectedTrashcan)
78 {
79     if (trash == null || trash.correctTrashcan == null)
80         return false;
81
82     // Tjek om skraldet hører til i den korrekte skraldespand
83     if (selectedTrashcan.ContainsTrash(trash))
84     {
85         return true;
86     }
87     else
88     {
89         return false;
90     }
91 }

```

Listing 6: Udsnit af TrashObject.cs scriptet fra #1 iteration

og i selve skraldespanden:

```

18 public bool ContainsTrash(Trash trash)
19 {
20     foreach (Trash t in correctTrash)
21         if (t.GetName() == trash.GetName())
22             return true;
23
24     return false;
25 }

```

Listing 7: Udsnit af Trashcan.cs scriptet fra #1 iteration

Koden sammenligner typen af skrald med indholdet i den valgte skraldspand. Hvis det matcher, får spilleren point, hvis ikke, mister spilleren et liv. Dette implementerer kravet (F1.06) om korrekt af-faldskontrol.

Spilleren belønnes med point, og highscore gemmes:

```

13 public void IncreaseScore(int scoreIncrease)
14 {
15     currentScore += scoreIncrease;
16
17     if (currentScore > highscore) highscore = currentScore;
18 }

```

Listing 8: Udsnit af PlayerScore scriptet fra #1 iteration

Dette gemmes i en CSV-fil:

```

37 using (StreamWriter writer = new StreamWriter(fullPath))
38 {
39     writer.WriteLine(highscore);
40     writer.Close();
41 }

```

Listing 9: Udsnit af PlayerScore.cs scriptet fra #1 iteration

Spillet holder løbende styr på både den aktuelle score og den højeste score nogensinde. Highscoren gemmes permanent i en CSV-fil og indlæses ved spilstart. Dette opfylder kravet (F1.07) om et vedvarende scoresystem.

Når spilleren sorterer forkert:

```

192 if (health <= 0)
193 {
194     Death();
195     return;
196 }
197
198 ...
199
200 else
201 {
202     health--;
203     UpdateHealth();

```

Listing 10: Udsnit af Game.cs scriptet fra #1 iteration

Liv reduceres ved forkert sortering, og når de løber ud, afsluttes spillet. Livene vises også som ASCII-kunst hjerter for tydelig visuel feedback. Dette opfylder kravet (F1.08) om et livs- og gameover-system.

4.1.2 Implementering af non-funktionelle krav

Ud over de funktionelle krav har vi også implementeret en række non-funktionelle krav, som vedrører spillets brugeroplevelse, robusthed, læsbarhed og tekniske kvalitet. Koden er udviklet med fokus på enkel og intuitiv navigation for unge brugere.

Vi anvender vores visuelle designfilosofi såsom farvemarkering af den valgte knap, centrerede menuer og faste layouts:

```
62 if (isHighlighted) Console.ForegroundColor = ConsoleColor.Green;
```

Listing 11: Udsnit af Button.cs scriptet fra #1 iteration

Dette betyder, at spilleren altid tydeligt kan se, hvor markøren er, og hvilke handlinger der er mulige. Derudover anvendes simple taster (pilene, Enter, Space) for at sikre, at styringen er let at forstå. Dette opfylder kravet (NF1.02) om at programmet skal kunne være nemt at bruge for unge i folkeskolen.

Koden understøtter læring gennem et feedbacksystem, der forklarer spilleren, hvorfor en sortering er forkert:

```
246 "" + currentTrash.trash.GetName() + "" + " hører til i " +  
247 currentTrash.trash.GetCorrect().GetName() + " skraldespandet.",
```

Listing 12: Udsnit af Game.cs scriptet fra #1 iteration

Her giver programmet en konkret tekstlig forklaring hver gang, spilleren laver en fejl. Dette gør, at brugeren gradvist lærer de korrekte sorteringsprincipper, uden at spillets flow afbrydes unødigt. Dette opfylder kravet (NF1.03) om at spilleren af programmet skal kunne lære noget om affaldssortering ved brug af spillet over en længere periode.

Teksten på skærmen er holdt i korte sætninger og vises i Unicode-symbolrammer, som tydeligt markerer menuer, infobokse og elementer på spilleskærmen:

```
270 Console.SetCursorPosition(boxX, boxY);  
271 Console.Write("┌");  
272 Console.SetCursorPosition(boxX + 1, boxY);  
273 Console.Write("─");  
274 Console.SetCursorPosition(boxX + 2, boxY);  
275 Console.Write("─");  
276 Console.SetCursorPosition(boxX + 3, boxY);  
277 Console.Write("─");  
278 Console.SetCursorPosition(boxX + 4, boxY);  
279 Console.Write("─");  
280 Console.SetCursorPosition(boxX + 5, boxY);  
281 Console.Write("└");
```

Listing 13: Udsnit af Game.cs scriptet fra #1 iteration

Dette gør det nemt for spilleren at se, hvilke elementer der hører sammen, og giver interfacet en genkendelig og logisk struktur. Samtidig er tekststørrelse og linjeskift nøje tilpasset konsolvinduets størrelse. Dette opfylder kravene (NF1.05) og (NF1.10) om teksten skal være læsbar, og størrelsen skal være passende, ligesom grafiske elementer skal være klare og intuitive for målgruppen.

4.2 #2 Iteration

Et af de grundlæggende mål med #2 iteration var at sikre, at spillet kunne køre uden fejl, der forstyrrede brugeroplevelsen (F2.01). Dette førte til en gennemgående oprydning i Game.cs, hvor inputhåndtering, tegning, tilstandsstyring og logik tidligere var blandet sammen. Hver funktion blev flyttet til sin egen klasse, eksempelvis InputManager, ConsoleDrawer og GameState, hvilket reducerede risikoen for fejl

og gjorde logikken mere overskuelig. Denne strukturering betyder, at potentielle bugs nu er isoleret til mindre moduler, hvilket gør både test og vedligehold væsentligt lettere.

I #2 iteration begyndte spillets verden også at få en reel progression. Et centralt krav var, at de fem levels ud over det første skulle være låst, indtil spilleren havde opdaget alle skraldetyper i det foregående level (F2.02). Dette blev implementeret direkte i World- og Space-klasserne:

```
9 private bool isLocked = true;
10
11 private readonly Space? nextLevel;
12
13 ...
14
15 public bool GetLockedStatus()
16 {
17     return isLocked;
18 }
19
20 public void UnlockLevel()
21 {
22     isLocked = false;
23 }
24
25 ...
26
27 public void DiscoverTrash(Trash trash)
28 {
29     if (undiscoveredTrash.Contains(trash))
30     {
31         int index = undiscoveredTrash.IndexOf(trash);
32         discoveredTrash.Add(undiscoveredTrash[index]);
33         undiscoveredTrash.RemoveAt(index);
34
35         // Add trash to Gamestate
36         Game.gameState.AddDiscoveredTrash(name, trash.GetName());
37     }
38
39     if (undiscoveredTrash.Count <= 0 && nextLevel != null)
40     {
41         nextLevel.UnlockLevel();
42     }
43 }
```

Listing 14: Udsnit af Space.cs scriptet fra #2 iteration

Her har hvert Space nu lister over kendt og ukendt skrald. Når spilleren sorterer korrekt, bliver det pågældende stykke skrald overført fra listen over ukendt skrald til listen over kendt skrald. Kun når listen med ukendt skrald rammer nul, bliver det næste level låst op.

```
49 if (Game.currentLevel.FollowEdge(Level.Levels[i]).GetLockedStatus())
50 {
51     buttons[i] = new Button(Level.levels[i], Theme.lockedColor, () =>
52         ↪ LevelLockedEvent(Level.levels[index]));
53     buttonToLevelSpaceMap[buttons[i]] = Game.currentLevel.FollowEdge(Level.levels[i]);
54     continue;
55 }
```

Listing 15: Udsnit af LevelSelectorScreen.cs scriptet fra #2 iteration

Her tjekkes låsestatus, når knapperne oprettes. Låste levels får en anden farve og viser en tekst, hvis man prøver at vælge dem. Dette sikrer, at spilleren først kan bevæge sig videre, når alle affaldstyper i et level er fundet.

I #2 iteration blev hvert level tildelt et unikt array af skrald (F2.03). Der oprettes et string-array med navnene på skrald for hvert level, som sendes gennem dictToList-metoden, der opretter nye Trash-

instanser ud fra navnene defineret i string-arrayet. På denne måde bliver arrayet af strings nemt konverteret til Trash-typer og brugt til at oprette de pågældende Spaces.

```
15     string[] hjemTrash =
16     {
17         "Ebleskrog",
18         "Bananskræl",
19         "Kaffefilter",
20         "Brødrester",
21         "Eggeskal",
22         "Papir",
23         "Avis",
24         "Toiletrulle",
25         "Pizzabakke med madrester",
26         "Plastflaske"
27     };
28
29     // Initialize levels each with their own list of items and fallSpeed
30     Space landLevel = new Space(Level.levels[5], dictToList(landTrash), 3d, null);
31     Space regionLevel = new Space(Level.levels[4], dictToList(regionTrash), 2.6d, landLevel);
32     Space kommuneLevel = new Space(Level.levels[3], dictToList(kommuneTrash), 2.2d, regionLevel);
33     Space strandLevel = new Space(Level.levels[2], dictToList(strandTrash), 1.8d, kommuneLevel);
34     Space byLevel = new Space(Level.levels[1], dictToList(byTrash), 1.4d, strandLevel);
35     Space hjemLevel = new Space(Level.levels[0], dictToList(hjemTrash), 1d, byLevel);
36     ...
37     private List<Trash> dictToList(string[] trashNames)
38     {
39         List<Trash> trashes = new List<Trash>();
40
41         for (int i = 0; i < trashNames.Length; i++)
42         {
43             trashes.Add(TrashTypes.trashDict[trashNames[i]]);
44         }
45
46         return trashes;
47     }
```

Listing 16: Udsnit af World.cs scriptet fra #2 iteration

Dette blev gjort ved at udvide Space-klassen, så den indeholder et sæt affaldstyper, der kun tilhører det pågældende område. Når et level indlæses, trækker TrashManager kun skrald fra det levels egen liste. Dette øger spillets variation, samtidig med at det støtter læringsmålet om at kende affaldstyper i kontekst.

Et andet vigtigt element var visualisering af progression i LevelSelector (F2.04). Spilleren skal kunne se, hvad der allerede er opdaget i hvert level, og hvor meget der stadig mangler.

```

161 // Display discovered items
162 Space hoveredSpace = buttonToLevelSpaceMap[currentSelectedButton];
163 List<Trash> discoveredTrash = hoveredSpace.GetDiscoveredTrashList();
164 List<Trash> undiscoveredTrash = hoveredSpace.GetUndiscoveredTrashList();
165
166 // Build a string of items to display.
167 string itemsToDisplay = "";
168
169 // Add discovered items first
170 foreach (Trash trash in discoveredTrash)
171 {
172     itemsToDisplay += trash.GetEmoji() + " " + trash.GetName() + " ";
173 }
174
175 // Add undiscovered items
176 foreach (Trash trash in undiscoveredTrash)
177 {
178     // Each undiscovered item should have its name bracketed out
179     string blockedName = "";
180     for (int i = 0; i < trash.GetName().Length; i++)
181     {
182         blockedName += "*";
183     }
184     itemsToDisplay += blockedName + " ";
185 }
186
187 // Display the items using LineWrapper
188 ConsoleDrawer.TextWrapper(maxTextLength, itemsToDisplay.TrimEnd(), startX, itemsY + 1, false,
189     ↪ Theme.foregroundColor);
190 }

```

Listing 17: Udsnit af LevelSelectorScreen.cs scriptet fra #2 iteration

Dette blev implementeret ved at udvide LevelSelectorScreen, så den læser et levels kendte og ukendte skrald, og fremviser dem i to grupper. Opdagede skrald samler alle trash's emojis og deres navne i én streng. Ukendt skrald samler, i samme streng, alle ukendte skrald, hvor hvert bogstav er erstattet med "*" symbolet. Herefter benyttes TextWrapper-metoden fra ConsoleDrawer-klassen, til at skrive det til konsollen.

I #2 iteration blev der også indført et nyt datasystem til at gemme og hente spillerens fremskridt på tværs af alle levels (F2.05). Ved hver iteration af gameloopet samles dataene for alle levels i et string-array, som derefter skrives til en fil.

```

26 public void Save()
27 {
28     string saveDataTitle = "LEVEL;HIGHSCORE;DISCOVERED";
29     string[] saveData = new string[Level.levels.Length + 1];
30
31     saveData[0] = saveDataTitle;
32
33     for (int i = 0; i < Level.levels.Length; i++)
34     {
35         string score = highscores[Level.levels[i]].ToString();
36         string discovered = string.Join("|", discoveredTrash[Level.levels[i]]);
37
38         saveData[i + 1] = $"{Level.levels[i]};{score};{discovered}";
39     }
40
41     DataHandler.Save(saveData.ToArray());
42 }

```

Listing 18: Udsnit af GameState.cs scriptet fra #2 iteration

Og i DataHandler ser selve gemme-metoden sådan ud:

```

10 public static void Save(string[] saveData)
11 {
12     string fullPath = Path.Combine(path, fileName);
13
14     StreamWriter writer = new StreamWriter(fullPath);
15
16     try
17     {
18         if (!File.Exists(fullPath))
19         {
20             File.Create(fullPath);
21         }
22
23         foreach (string data in saveData)
24         {
25             writer.WriteLine(data);
26         }
27
28         writer.Close();
29     }
30     finally
31     {
32         writer.Close();
33     }
34 }

```

Listing 19: Udsnit af DataHandler.cs scriptet fra #2 iteration

Ved indlæsning bruger GameState.Load de samme strukturer til at genskabe highscores og markere allerede opdaget skrald via DiscoverTrash på de enkelte Space-objekter. På den måde bliver progressionen permanent, og spilleren kan vende tilbage til et spil, hvor både highscore og opdagede skraldtyper for hvert level bevarer.

Den sidste større funktionelle ændring i #2 iteration var introduktionen af gradvist opdaget skrald (F2.06). Tidligere blev nyt skrald valgt fuldstændig tilfældigt fra hele et levels liste, men nu bliver ukendte skrald introduceret i intervaller via TrashManager.UnlockTrash. Det betyder, at spilleren ikke møder alle typer på én gang, men at nye typer langsomt bliver en del af spillet i takt med, at spilleren scorer point.

```

88 public static void UnlockTrash()
89 {
90     if (Game.currentLevel != null && (Game.currentLevel.GetDiscoveredTrashList().Count <
91     ↪ GameConstants.instantDiscoverRate || Game.gameState.GetCurrentScore() >
92     ↪ GameConstants.scoreScaling * Game.unlockCount))
93     {
94         Game.unlockCount++;
95
96         if (undiscovered.Count > 0)
97         {
98             revealThis = undiscovered[rand.Next(0, undiscovered.Count)];
99             Game.currentLevel.DiscoverTrash(revealThis);
100
101             Game.discoveryMessage = "Nyt skrald opdaget: " + revealThis.GetName();
102             Game.messageEndTime = DateTime.Now.AddSeconds(3);
103
104             currentTrash.SetTrash(revealThis);
105             revealThis = null;
106         }
107     }
108 }

```

Listing 20: Udsnit af TrashManager.cs scriptet fra #2 iteration

Funktionen UnlockTrash i TrashManager.cs er ansvarlig for gradvist at frigive nye skraldtyper i spillet. Når spilleren enten har opdaget for få typer skrald (instant discover) eller har opnået tilstrækkelig score, vælger funktionen et tilfældigt stykke ukendt skrald, markerer det som opdaget og indsætter det som

det næste faldende skrald-objekt. Dette implementerer fuldt ud krav (F2.06) om gradvis introduktion af nyt skrald.

#2 iteration udgjorde dermed et vigtigt skridt fra et basalt, fungerende spil til et mere professionelt system med fokus på progression, stabilitet og læring. Gennem refaktoring, modulopdeling, datalagring og dynamisk affaldsintroduktion er spillet blevet langt mere robust og sammenhængende, samtidig med at brugeroplevelsen er styrket markant.

5 Test af koden - NUnit tests

Under udviklingen af vores spil introducerede vi NUnit til at enhedsteste dele af logikken. Vi har ikke testet hele spillet, da meget af funktionaliteten afhænger af konsol-output, timing og grafik, men enkelte kernefunktioner kan verificeres med tests.

Test af Fall-metoden Et eksempel på ovenstående er testen af Fall-metoden i TrashObject-klassen. Metoden styrer, hvor langt et stykke skrald falder ned ad skærmen. En vigtig del af logikken er, at objektet aldrig må falde under spillets nederste grænse (defineret i "GameConstants.gameHeight"). Her er implementationen fra spillet:

```
49 public void Fall(int fallAmount)
50 {
51     positionY += fallAmount;
52
53     if (positionY >= GameConstants.gameHeight)
54         positionY = GameConstants.gameHeight;
55 }
56
57 public void Fall(int fallAmount, bool instant)
58 {
59     if (instant)
60         positionY = GameConstants.gameHeight;
61     else
62         Fall(fallAmount);
63 }
```

Listing 21: Fall-metoden fra TrashObject.cs

For at dokumentere denne metode har vi lavet en NUnit-test, der verificerer, at metoden korrekt stopper objektet ved bunden af skærmen, selv hvis "fallAmount" er meget større end normalt. Det sikrer, at objektets position aldrig kan ende i en ugyldig tilstand.

```
14 [Test]
15 public void Fall_ShouldNotExceedGameHeight()
16 {
17     Trash trash = new Trash("Test"); // Test trash
18     TrashObject obj = new TrashObject(trash);
19
20     obj.Fall(200); // A big fall, which would exceed the screen dimensions
21
22     Assert.That(obj.positionY, Is.EqualTo(GameConstants.gameHeight));
23 }
```

Listing 22: Test kode til Fall-metoden

Resultat

Testen består, som ses på Figur 9. Dette viser, at vores implementation af Fall indeholder en korrekt grænsekontrol, så skraldet ikke bevæger sig ud over spillets definerede grænser. Dette er vigtigt, selvom skraldet normalt bliver samlet op af en skraldespand før det når bunden. Testen demonstrerer, at vi sikrer stabil opførsel uanset input.

```
Running tests...
✅ PASSED: Fall_ShouldNotExceedGameHeight (5ms)
=====
Test Run Summary
=====
Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: ✅ SUCCEEDED
Test Execution Time: 0.09s (91ms)
Total Time (including build): 1.30s (1305ms)
=====
```

Figur 9: Resultat af Unit Test til Fall-metoden.

Test af korrekt sortering af skrald

Denne test verificerer, at et stykke skrald bliver sorteret rigtigt, når det lander i den korrekte skraldespand. Hvert Trash-objekt har en reference til den skraldespand, det hører til, og CheckSorting-metoden skal returnere true, hvis spilleren sorterer korrekt. Testen opretter et skrald-objekt og en skraldespand, som matcher hinanden, og kontrollerer derefter, at sorteringen accepteres.

```
25 [Test]
26 public void CheckSorting_CorrectTrashCan()
27 {
28     Trash testTrash = new Trash("Test_1");
29
30     Trash[] correctTrashes =
31     [
32         testTrash
33     ];
34
35     Trashcan testTrashcan = new Trashcan("TrashcanTest", correctTrashes);
36
37     testTrash.correctTrashcan = testTrashcan;
38
39     TrashObject obj = new TrashObject(testTrash);
40
41     bool result = obj.CheckSorting(testTrashcan);
42
43     Assert.That(result, Is.True);
44 }
```

Listing 23: Test af korrekt sortering af skrald

Test af forkert sortering af skrald

Modsat den forrige test verificerer denne test, at spillet afviser forkert sortering. Her oprettes to forskellige skraldespande med hver deres type skrald. Et stykke skrald forsøges placeret i en forkert skraldespand, og metoden forventes at returnere false.

```

46 [Test]
47 public void CheckSorting_WrongTrashCan()
48 {
49     Trash correctTrash = new Trash("Correct");
50
51     Trash[] correctTrashes =
52     [
53         correctTrash
54     ];
55
56     Trash wrongTrash = new Trash("Wrong");
57
58     Trash[] wrongTrashes =
59     [
60         wrongTrash
61     ];
62
63     Trashcan trashcan_1 = new Trashcan("Trashcan 1", correctTrashes);
64     Trashcan trashcan_2 = new Trashcan("Trashcan 2", wrongTrashes);
65
66     correctTrash.correctTrashcan = trashcan_1;
67     wrongTrash.correctTrashcan = trashcan_2;
68
69     TrashObject obj = new TrashObject(wrongTrash);
70
71     bool result = obj.CheckSorting(trashcan_1);
72
73     Assert.That(result, Is.False);
74 }

```

Listing 24: Test af forkert sortering af skrald

Denne test sikrer, at spillets sorteringslogik ikke giver point for forkert adfærd, hvilket er essentielt for spillets gameplay.

Test af score og highscore funktionalitet

Denne test fokuserer på GameState-klassen, som håndterer spillerens score og highscores pr. level. Testen kontrollerer flere scenarier i en sammenhængende test:

- At scoren starter på 0
- At scoren øges korrekt
- At highscore opdateres, når scoren overstiger den tidligere highscore
- At den nuværende score nulstilles uden at highscore mistes
- At highscore ikke opdateres, hvis den nye score er lavere

```

76 [Test]
77 public void GameState_ScoreFunctionalityTest()
78 {
79     GameState gameState = new GameState();
80
81     string level = Level.levels[0];
82
83     int initialScore = gameState.GetCurrentScore();
84     int initialHighscore = gameState.highscores[level];
85
86     Assert.That(initialScore, Is.EqualTo(0));
87     Assert.That(initialHighscore, Is.EqualTo(0));
88
89     int scoreIncrease = 100;
90
91     gameState.IncreaseScore(level, scoreIncrease);
92
93     int increasedScore = gameState.GetCurrentScore();
94
95     Assert.That(increasedScore, Is.EqualTo(scoreIncrease));
96
97     int updatedHighscore = gameState.highscores[level];
98
99     Assert.That(updatedHighscore, Is.EqualTo(scoreIncrease));
100
101     gameState.ResetScore();
102
103     int resetCurrentScore = gameState.GetCurrentScore();
104     int resetCurrentHighscore = gameState.highscores[level];
105
106     Assert.That(resetCurrentScore, Is.EqualTo(0));
107     Assert.That(resetCurrentHighscore, Is.EqualTo(scoreIncrease));
108
109     int newScoreIncrease = 50;
110
111     gameState.IncreaseScore(level, newScoreIncrease);
112
113     int newCurrentScore = gameState.GetCurrentScore();
114     int newCurrentHighscore = gameState.highscores[level];
115
116     Assert.That(newCurrentScore, Is.EqualTo(newScoreIncrease));
117     Assert.That(newCurrentHighscore, Is.EqualTo(scoreIncrease));
118 }

```

Listing 25: Test af score og highscore funktionalitet

Testen dokumenterer, at spillets point-system er stabilt og ikke utilsigtet overskriver highscores.

Test af level-navigation (eksisterende edge)

Spillet benytter en grafstruktur (Space) til at navigere mellem levels og menuer. Denne test verificerer, at det er muligt at følge en eksisterende forbindelse (edge) mellem to rum.

```

124 [Test]
125 public void FollowEdge_ExistingDirection()
126 {
127     string levelName = "Level 1";
128
129     Space entry = new Space("Entry", new List<Trash>(), 0d, null);
130     Space level_1 = new Space(levelName, new List<Trash>(), 0d, null);
131
132     entry.AddEdge(levelName, level_1);
133
134     Space loadedLevel = entry.FollowEdge(levelName);
135
136     Assert.That(loadedLevel.GetName(), Is.EqualTo(levelName));
137 }

```

Listing 26: Test af level-navigation (eksisterende edge)

Testen viser, at level-navigationen fungerer korrekt, når spilleren vælger et gyldigt level.

Test af level-navigation (ikke-eksisterende edge)

Den sidste test undersøger et grænsetilfælde, hvor spilleren forsøger at navigere til et level, som ikke findes. I dette tilfælde forventes systemet at kaste en undtagelse (exception).

```
140 [Test]
141 public void FollowEdge_NonExistingDirection()
142 {
143     string levelName = "Level 1";
144
145     Space entry = new Space("Entry", new List<Trash>(), 0d, null);
146     Space level_1 = new Space(levelName, new List<Trash>(), 0d, null);
147
148     entry.AddEdge(levelName, level_1);
149
150     Assert.Throws<KeyNotFoundException>(() => entry.FollowEdge("NonExistingDirection"));
151 }
```

Listing 27: Test af level-navigation (ikke eksisterende edge)

Denne test dokumenterer, at systemet fejler kontrolleret og forudsigeligt ved ugyldig navigation, hvilket øger robustheden af spillets struktur.

Bemærkning Testene demonstrerer altså, hvordan man med NUnit kan validere robustheden af enkelte metoder, også selv om den konkrete situation sjældent forekommer i selve gameplayet.

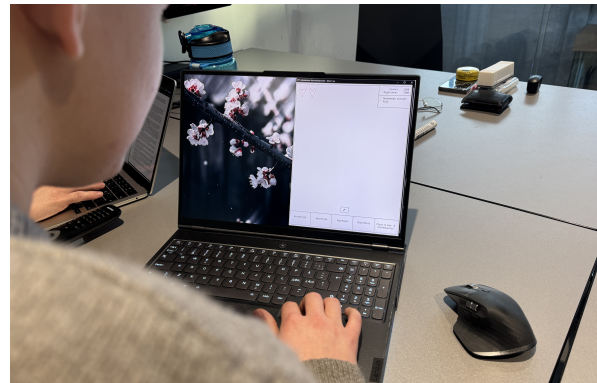
6 Test af løsning

For at vurdere effekten af produktet, blev der udført en test af løsningen. Denne test blev først udført i #2 Iteration af projektet, efter implementeringen af ny funktionalitet. Først blev der formuleret en plan for udførslen af testen. Denne plan er bl.a. baseret på feedback modtaget fra midtvejsseminaret, hvoraf det blev konkluderet, at spillet bedst kunne testes ved et indledende spørgeskema for at sætte et grundlag for testpersonens nuværende forståelse af og forvirring omkring emnet.

Spørgsmål som "Hvornår var du senest i tvivl om hvordan du skulle sortere noget affald?" og "Hvor tit er du i tvivl om hvordan du skal sortere noget affald?" er med til at få et overblik over testpersonens egen vurdering. Yderligere bliver der stillet case-spørgsmål som "Hvordan ville du sortere en brugt og tømt mælkekarton?" eller "Hvordan ville du sortere en tom og ren konservesdåse?". På denne måde afklares testpersonernes egen vurdering, samtidig med at det giver indblik i deres faktiske forståelse ud fra case-spørgsmålene.

Ved besvarelse af de indledende spørgsmål var det synligt, at størstedelen af testpersonerne har været usikker på affaldssorteringen indenfor de seneste seks måneder. Dette understøtter vores antagelse af, at det er et reelt problem for unge i vores målgruppe. Dog svarede størstedelen af personerne korrekt ved vores case-spørgsmål, som er en tydelig indikation på, at vores case-spørgsmål skal revideres.

Efter besvarelse af de indledende spørgsmål, blev alle testpersoner bedt om at spille spillet. Vi prøvede så vidt muligt ikke at give hints eller forklaringer, så vi kunne få en fornemmelse af testpersonernes organiske oplevelse. Under spillet kommenterede flere testpersoner bl.a. på, at de ikke vidste, hvordan de skulle spille, og hvad de forskellige elementer var - hvor vi blev nødt til at gribe ind og give en forklaring på dette. Der var en generel tendens til at testpersonerne udviklede sig gennem spillet, da



Figur 10: Billede fra test af programmet.

der var en tydelig progression mellem de runder, de spillede. De fleste testpersoner fik en bedre score ved andet forsøg.

Efter at testpersonerne havde været gennem spillet, udførte vi endnu en spørgerunde, hvor vi spurgte ind til deres oplevelse af spillet. F.eks. blev der spurgt ”Føler du, at du har fået noget ud af dette spil?” eller ”Hvad tror du formålet med spillet er?”. Dette blev gjort med hensigten at få deres indledende tanker, samt styre en mindre struktureret samtale om spillet efterfølgende.

En bemærkelsesværdig besvarelse er:

Spørgsmål: Føler du, at du har fået noget ud af dette spil?

Svar: Nej, det er alt for simpelt og nemt til min alder.

Flere testpersoner fra den ældre ende af målgruppen havde samme holdning, som viser at projektets målgruppe måske har været for bred. Én testperson kommenterer, at:

”[...] man kan ikke sammenligne en 19-årig med en 10-årig”

Dette er en relevant overvejelse, gruppen også havde, men vi valgte at fokusere på andre relevante faktorer til at definere vores målgruppe. Men ud fra tendensen af resultaterne fra vores tests, kan det være relevant at genoverveje dette.

Derudover har vi observeret, at den generelle opfattelse af spillet er, at det er kedeligt, og de nok ikke ville spille spillet i deres fritid. Dog forklares det efter opfølgende spørgsmål, at hvis valget var mellem normal skole eller dette spil, ville størstedelen af vores testpersoner vælge dette spil.

Ydermere udtrykkes der potentiale for spillet, men i dets nuværende stadie vil det ikke have den største effekt. En større del af testpersoner udtrykker også, at de har fået lidt til ingenting ud af spillet. Dette kan dog være grundet den begrænsede tid med spillet, da testpersoner som oftest ikke spillede mere end to gange og ikke længere end 5-15 minutter.

En anden overvejelse der blev gjort i løbet af testene, var diagnoser som ordblindhed, der havde en særdeles negativ effekt på testpersonens præstation. Dette skyldes begrænsningen af grafik, der gør at skrald bliver repræsenteret af tekst. Et potentielt forbedringspunkt til en senere udvikling af projektet kunne være at implementere relevante grafiske elementer, for at mindske mængden af tekst.

7 Evaluering

7.1 Iteration 1

Vi havde en række forventninger til projektet: Løsningen skulle være let at bruge, visuelt tiltalende og indeholde tydelige forklaringer og eksempler på sortering. Programmet kunne motivere brugerne til at lære gennem simple, intuitive funktioner og præsentere information på en måde, der var både forståelig og relevant for målgruppen. Vi som gruppe ville kunne nå implementeringen af kernefunktionerne gennem #1 iteration og derefter bruge #2 iteration til at forbedre brugeroplevelsen og rette fejl, der opstod undervejs.

I #1 iteration nåede vi det meste af det, vi havde sat som mål. Vi fik gjort både de funktionelle og non-funktionelle krav færdige, men ikke alle vores idéer og formål blev opnået endnu. Et konkret eksempel er, at vi fandt ud af, at vores levels i spillet skal være låst, så spilleren ikke starter i det sværeste level. Vi manglede også at bygge levels mere meningsfuldt op i forhold til, hvad der bliver sorteret i hvert område. For eksempel havde vi ideen om, at i ”hjemme”-området finder man bestemte typer skrald, mens man i ”by”-området finder både det og mere.

En stor læringsoplevelse har været, hvordan man strukturerer et større projekt. Det er både en iterativ proces og en læringsproces. Vi bruger GitHub, hvilket giver mulighed for at se tidligere versioner af vores arbejde, så på den måde kan vi følge vores udvikling og lære af vores fejl.

I forhold til #1 iterations krav er F1.01–F1.09 alle blevet færdiggjort. De dækker primært basen for vores spil: At vise skærme, lade spilleren navigere rundt, flytte skrald, sortere det og optjene point.

Det nuværende produkt har mod forventning stadig fejl, bugs og forbedringspunkter. Derfor er et mål for #2 iteration at udrede dette og forbedre den overordnede spiloplevelse. Ydermere er kravene

NF1.06–NF1.07 ikke blevet opnået fuldt ud. Dette bliver f.eks. beskrevet af vores ”rodede” kodebase og generelle kritiske fejl i programmet. Dermed er et mål for #2 iteration at opnå dette, samt øge programmets tolerance for fejl og forbedre kodebasens struktur.

7.2 #2 Iteration

#2 Iteration er nødvendig for, at vi kan finalisere vores projekt. Som nævnt tidligere, var der i #1 iteration stadig forbedringspunkter, der blev arbejdet på gennem #2 iteration. Programmet blev lagdelt, og kodebasen blev refaktoreret.

Ifølge kravene opstillet for #2 iteration, er programmet næsten fuldført. Enkelte kritiske fejl præger stadig kodebasen, som gør, at programmet ikke lever op til krav F2.01. Ydermere kan der være specifikke forbedringer til UI, forklaringer eller andet, der ville gøre programmet mere komplet. Reflekteres der over disse mangler kan det argumenteres, at programmet for det meste lever op til forventningerne. Dette efterlader også flere forbedringspunkter der kan arbejdes videre på i fremtiden.

Samlet set vurderes processen af #2 iteration til at være succesfuld, og på mange måder har løftet projektet markant i forhold til #1 iteration. Løsningen fremstår nu mere sammenhængende, struktureret og tættere på de mål fastsat for projektet. De funktionelle krav er stort set opfyldt, og brugeroplevelsen er blevet forbedret. Mere variation i skrald og en mere informativ menu understøtter den læring, der forventes af spillet. Selvom der stadig kan være mindre forbedringer, særligt i forhold til UI og fremtidig udvidelse, mener vi, at projektet i sin nuværende form lever op til vores forventninger for #2 iteration. Iterationen har vist, at de valg og prioriteringer, der blev truffet på baggrund af #1 iteration, har haft en positiv effekt, og at projektet nu befinder sig på et niveau, vi som gruppe er tilfredse med.

8 Konklusion

I dette projekt blev der stillet spørgsmålet ”Kan vi øge viden om korrekt affaldssortering hos unge fra 10-19 år i Danmark og dermed mindske forkert affaldssortering og øge genanvendelsen af vigtige ressourcer, gennem et læringsspil med fokus på trial & error princippet?”

På baggrund af vores evaluering kan vi konkludere, at vi har opnået projektets krav. Vi har fuldført alle krav stillet af opgaven, og de krav, vi selv har stillet, i vores funktionelle og non-funktionelle krav, både i #1 og #2 iteration af processen. Dermed er der blevet produceret et læringsspil, der opfylder formålet opstillet i problemformuleringen.

Selvom det indikeres ud fra vores test af løsningen, at produktet har potentiale, vurderes det, at den nuværende tilstand af produktet ikke opfylder problemformuleringens indledende formål om at øge viden om korrekt affaldssortering fyldestgørende. Dermed vil resultatet ikke føre til en synlig reduktion af forkert affaldssortering. Det konkluderes dermed, at svaret på problemformuleringen er nej.

9 Procesevaluering

9.1 Etablering af samarbejdsgrundlag

Allerede i projektets tidlige fase udarbejdede gruppen en samarbejdsaftale, som fastlagde fælles forventninger til mødedeltagelse, arbejdsindsats, kommunikation og håndtering af konflikter. Aftalen beskrev også, hvordan gruppen skulle forholde sig til sygdom, fravær og generelle udfordringer, der kunne opstå undervejs. Samtidig blev der udarbejdet en vejlederaftale, hvor gruppen og vejlederen afstemte rammerne for møder, krav til forberedelse og forventet kommunikation. Gruppens roller, såsom referent, ordstyrer og kontaktperson til vejleder, blev løbende roteret for at give alle erfaring med forskellige ansvarsområder.

9.2 Arbejdsform og metoder

Projektarbejdet blev organiseret efter en iterativ arbejdsproces, hvor design, implementering og evaluering blev udført i flere omgange. En central del af gruppens arbejdsform var rotationsprincippet,

hvor hvert gruppemedlem fik mulighed for at arbejde med forskellige typer af opgaver. Dette sikrede, at alle fik indsigt i både analyseværktøjer, designmetoder, programmering og dokumentation. Gennem hele projektet anvendte gruppen blandt andet problemtræ, noun/verb-analyse, CRC-kort og UML-diagrammer. Rotationen af opgaver betød, at alle deltog i udviklingen af hver delmetode, hvilket skabte et fælles fagligt fundament for implementeringsfasen.

9.3 Kommunikation og koordinering

Kommunikationen i projektet foregik primært gennem Messenger, hvor gruppen koordinerede møder, afstemte mails til vejlederen og informerede om sygdom eller fravær. Discord blev anvendt som platform til online møder og deling af links, når gruppen ikke mødtes fysisk. Deling af dokumenter, præsentationer og øvrige projektrelaterede materialer blev håndteret via en Google Drive-fællesmappe. Denne tydelige opdeling af kommunikationskanaler gjorde det nemt at holde overblik og sikre effektiv koordinering gennem hele projektforløbet. Hvis et gruppemedlem var fraværende ved et møde, sørgede gruppen for at holde den pågældende opdateret gennem korte opsummeringer eller online møder, inden næste trin i processen blev påbegyndt.

9.4 Udviklingsværktøjer og teknisk samarbejde

Til den tekniske del af projektarbejdet anvendte gruppen GitHub som fælles repository og versionsstyringssystem. Det gjorde det muligt at arbejde parallelt på forskellige dele af projektet, samtidig med at alle ændringer blev registreret og versioneret. For at undgå konflikter i koden blev arbejdsopgaverne organiseret efter klasser, hvilket var muligt på baggrund af UML-diagrammerne. Når gruppen arbejdede fysisk sammen, blev Live Share i Visual Studio Code brugt til at skrive kode i realtid i samme session. Denne arbejdsform skabte et effektivt fælles udviklingsmiljø, hvor idéer, rettelser og forbedringer kunne implementeres på stedet. Efter den samlede implementering gennemgik gruppen kodebasen for at sikre fælles forståelse og sammenhæng.

9.5 Dokumentationsværktøjer og andre værktøjer

Til rapport, logbog og supplerende dokumenter anvendte gruppen Word og Google Drive. Logbogen blev skrevet i Word og gemt direkte i GitHub-repositoriet, så både kode og procesdokumentation var samlet et sted. Selve rapporten blev udarbejdet i Word, men gruppen planlagde at overføre rapporten til LaTeX på et senere tidspunkt, da det forventes at blive brugt i kommende semestre. Til præsentationer blev der anvendt PowerPoint, mens Canva blev brugt til vores projekt-poster.

9.6 Tidsstyring og arbejdsfordeling

Gruppens arbejdsfordeling byggede på både kompetencer og interesser, men rotationsprocessen gjorde, at ingen var begrænset til én type opgave. Tidsstyringen blev håndteret gennem faste ugentlige møder, hvor der blev aftalt deadlines og arbejdsplaner for de kommende dage. Derudover sikrede den løbende kommunikation, at eventuelle forsinkelser eller ændringer hurtigt kunne håndteres. Gruppen formåede gennem hele projektforløbet at overholde de fleste deadlines og opretholde en stabil arbejdsindsats.

9.7 Læringsprocessen

Projektet gav gruppen en dybere forståelse af objektorienteret analyse og design, herunder anvendelse af CRC-kort, UML-diagrammer og kravspecifikationer. Derudover opnåede gruppen praktisk erfaring med GitHub, Live Share og generelt samarbejde om koden. Projektet gav også indsigt i iterative arbejdsmetoder, dokumentationskrav og teknisk formidling i en rapport. Rotation af roller og opgaver viste sig at være særligt værdifuld, da det gav alle et bredt indblik i hele udviklingsprocessen fra idé til færdigt produkt.

9.8 Refleksion over samarbejdet

Samarbejdet i gruppen har fungeret meget tilfredsstillende. Alle medlemmer bidrog aktivt og konstruktivt, og kommunikationen var både åben og effektiv. Gruppens håndtering af fravær, koordinering og

deling af ansvar har spillet en central rolle i et succesfuldt projektforsløb. I gruppen er vi samlet set tilfredse med vores samarbejde, produkt og det faglige udbytte, som projektet har genereret.

9.9 Forbedringspunkter og erfaringer til fremtidige projekter

I vores samarbejdsaftale havde vi formuleret, at aftalen skulle tages op til revision ved afslutningen af hver projektfase. Formålet var at reflektere over spørgsmålene: *Hvad aftalte vi? Hvad gjorde vi? Hvad skal vi fortsætte med, stoppe med eller begynde på?* Selvom dette princip var en del af aftalen, endte gruppen med ikke at gennemføre en systematisk evaluering ved hver faseafslutning. Dette påvirkede dog ikke projektet negativt, da alle gruppemedlemmer løbende fulgte samarbejdsaftalen og bidrog ansvarligt gennem hele forløbet.

Derudover har gruppen erfaret, at det kan være uhensigtsmæssigt at foretage for mange ændringer i projektets struktur og løsninger undervejs, efterhånden som nye metoder og teknikker introduceres i undervisningen. I dette projekt medførte de løbende ændringer uklarheder i relation til kodebasen, hvor det var svært at holde styr på, hvad der var ændret. Dette var et vilkår ved den sideløbende undervisning på universitetet. En stærkere og mere konkret plan i starten af projektet ville have gavnet forløbet. Gruppen arbejdede dog aktivt for at minimere disse udfordringer ved at præsentere og gennemgå ændringer ved hvert møde.

9.10 Brug af generativ AI-værktøjer

AI-værktøjer blev benyttet gennem projektets faser til idégenerering, brainstorming, research, og grammatiske rettelser.

AI-værktøjer er blevet anvendt som et støtteværktøj gennem hele projektets arbejdsproces. I den indledende fase blev AI brugt til idégenerering og brainstorming ved at foreslå mulige problemstillinger, underemner og strukturer, som kunne danne udgangspunkt for projektets videre arbejde. Dette bidrog til at afgrænse projektets fokus og skabe overblik over relevante tilgange. Under udarbejdelsen af rapporten blev det anvendt til sproglig støtte, herunder grammatiske rettelser, forslag til alternative formuleringer samt forbedring af tekstens struktur og sammenhæng.

Litteratur

- [1] D. Statistik, "Husholdningerne sorterer affald i stigende grad," Sept. 2025.
- [2] F. A. Zeuthen, "Fire ud af ti sorterer affald forkert," Sept. 2021.
- [3] S. K. Galán, "Bliver affaldet bare blandet sammen? få svarene på 3 betænkeligheder ved affaldssortering her," Aug. 2022.
- [4] F. A. Zeuthen, "Én ud af tre danskere: Det er for besværligt at sortere affald," Sept. 2021.
- [5] Miljøministeriet, "Survey om affaldssortering," Jan. 2022.
- [6] A. Brereton, "Affaldssortering og nudging."
- [7] Dansk Erhverv, "De fleste danskere sorterer deres affald," Dec. 2022.
- [8] M. H. Mortensen, "Ny affaldsstatistik gør os ikke klogere på plastikaffald," Jan. 2023.
- [9] P. Petersen, "Tilpas lektielæsningen ved at forstå dit barns læringsstil," Feb. 2020.
- [10] C. Dreier, "Læringsstile - hvad er det?," Dec. 2021.
- [11] L. Christofferse, "Hvordan styrker vi positive fællesskaber blandt børn og unge?," Nov. 2019.
- [12] S. Hagemann, "Ny viden om teenagehjernen er fantastisk nyt for pædagoger," Apr. 2023.
- [13] Toptutors, "10 råd til at motivere dit barn i skolen."
- [14] J. Piaget, "Jean piagets teori om kognitiv udvikling."
- [15] R. Puche, "Trial and error: the god complex," Oct. 2025.

- [16] T. D. Henriksen, “læringsspil,” Oct. 2024.
- [17] OOP Handout, “Oop handout: Object-oriented software development,” Oct. 2025. Handout, October 6th, 2025.

10 Bilag

10.1 Samarbejdsaftale

Dette er gruppens Samarbejdskontrakt:

Samarbejdsaftale (Gruppekонтракт)

Forventninger og mål for projektet

1. Det er et fælles mål blandt gruppemedlemmerne at blive bedre til fagenes indhold. Alle ønsker, at alle får en forståelse for hele projektet og relevant materiale, så vi fælles kan komme til afslutningen af projektet med samme niveau.
2. Gruppemedlemmerne bestræber sig så vidt muligt at fremstille et projekt som alle er tilfredse med.
3. Gruppens medlemmer skal så vidt muligt gøre deres bedste i alle aspekter af projektets omfang. Det taget i betragtning, kan der ikke forventes yderligere udover ens bedste mht. den enkeltes kompetencer og niveau.
4. Gruppens medlemmer har fælles forhåbninger om at klare sig godt i faget og dets eksamen. Ingen fast karakter grænse påkrævet, men gruppen stræber efter at gøre sit bedste.
5. Det forventes, at alle medlemmer af gruppen leverer det bedst mulige materiale, når den pågældende person har fået en opgave. Herunder forventes det, at materialet er udarbejdet til tiden, der er læst korrektur, materialet er gjort tilgængeligt for gruppen vha. gruppens fil-delingsværktøj, mv.

Hvad motiverer den enkelte i gruppen til at gøre en indsats?

1. Ansvarsfølelsen over for de andre gruppemedlemmer gør, at man yder en bedre præstation.
2. At vi får udarbejdet et projekt vi er stolte af. Hvor vi alle er enige om at der har været en stor indsats og arbejdet er gjort ordentligt.

Arbejdstider

1. Hver tirsdag mødes vi lidt før 8:00 og arbejder indtil 13:00 - som udgangspunkt arbejdes der yderligere indtil klokken 15:00, men såfremt et gruppemedlem har andre vigtige forpligtelser, har de mulighed for dette fra klokken 13:00. Det forudsættes, at der er tale om uopsættelige aftaler, som ikke kunne placeres uden for arbejdstiden (såsom læge, tandlæge, eller andet vigtigt). Det aftales med flertal i gruppen, om en pågældende aftale er begrundet.

2. Tirsdag aftaler vi et tidspunkt inden det kommende vejledermøde, hvor vi sætter os i 15-30 minutter og får et overblik over dagsordenen, hvem der sender invitationen og hvilke spørgsmål vi har.
3. Hver fredag arbejdes der fra 8:00-12:00* - Undtagelser kan forekomme ved enstemmigt valg om arbejdet er nødvendigt eller ej.
4. Såfremt der aftales individuelt arbejde eller hjemmearbejde, forventes det, at man selv afsætter nødvendigt tid til at udføre opgaverne til den aftalte deadline.

*med visse undtagelser mht. transport til studie.

Gruppemøder

1. Er et gruppemedlem forhindret i at møde til de aftalte tidspunkter, skal vedkommende ved tidligst mulige lejlighed informere gruppen om dette.

Organisering af møder

1. Til hvert vejledermøde får et gruppemedlem rollen som dirigent (ordstyrer) og en anden får rollen som beslutningsreferent. Rollerne som dirigent og referent følger det forud fremstillede skema.
2. Således angives rollen for gruppemedlemmer med et X. Efter hvert vejledermøde rykker X'erne en plads ned. X'erne er forskudt med 2, så du ikke varetager en rolle flere møder i træk.

Gruppemedlem	Dirigent	Referent / Logbog
Aliza		
Mikkel Blom	X	
Octavian		
Yasser		
Ziad		X

3. Såfremt nødvendigt ved sygdom, fravær eller andet, videregives rollen til det næste fremmødte medlem, som ikke allerede varetager en rolle. Den/de fraværende påtager sig den pågældende rolle af gruppemedlemmet som overtog for dem, i næste cyklus af roller.*
4. Hvert eneste møde føres der en logbog over mødets vigtigste punkter, beslutninger, mv. Dette inkluderer både vejledermøder og almene arbejds møder. Hver arbejdsdag (dvs. en aftalt dag, hvor vi alle fælles eller individuelt arbejder på projektet) betragtes

som et arbejds møde og kræver dermed føring af en logbog. Logbogen føres af referenten.

*Eksempel: Hvis Octavian er syg og Mikkell ikke møder op, når det er Mikkells tur til at være dirigent, så overtager Yasser Mikkells rolle som dirigent. Når det så er Yassers tur til at være dirigent, vil det være Mikkell, som skal overtage rollen. Herefter genoptages den normale cyklus af roller.

Fordeling af fælles og individuelt arbejde

1. Hver tirsdag mødes vi sammen på skolen fra 8:00-13:00, for at arbejde på projektet. Gruppens medlemmer sætter sig sammen og arbejder. Enten arbejder vi alle gruppemedlemmer, eller også opdeles arbejdet enkeltvis eller i mindre grupper.
2. Hver fredag fra 8:00-12:00* arbejdes der efter aftale enten: 1) Individuelt, hvor gruppens medlemmer stiller sig tilgængelige online i det angivet tidsrum. 2) Fælles, hvor vi mødes fysisk og arbejder sammen i det angivet tidsrum. 3) Fælles, hvor vi mødes online og arbejder sammen i et taleopkald i det angivet tidsrum. Såfremt der er fælles enighed om det blandt gruppens medlemmer, kan dette fredagsmøde flyttes eller aflyses.

*med visse undtagelser mht. transport til studie.

Arbejdsfordeling af rapportskrivning

1. Alle rapportens afsnit fordeles ligeligt blandt gruppens medlemmer for at sikre effektivt og ligeligt arbejde. Fordelingen af afsnit foretages løbende efter aftale med gruppens medlemmer.
2. Rapporten skrives sideløbende med projektet. Dvs. at gruppens medlemmer løbende foretager tilføjelser og ændringer til rapporten, for at sikre mere tid i slutningen af projektet.

Vejledermøder

1. Der afholdes ét vejledermøde hver tirsdag, hvor vi har mulighed for spørgsmål, samt sikre at vi er på rette spor.
2. Yderligere vejledningsmøder aftales efter behov i samarbejde med vejleder.
3. Gruppen har pligt til at sende mødeindkaldelse med lokale, dagsorden, tidspunkt og relevante materialer via outlook til vejlederen senest kl. 23:59 dagen før materialet skal behandles.

Kursusdeltagelse & opgaveløsning

1. Der forventes ikke, at gruppens medlemmer møder op til forelæsninger og øvelsestimer. Dette vurderes individuelt efter behov.
2. Så vidt muligt skrives kommentarer i koden for at gøre den så forståelig som mulig.
3. Det forventes, at hvert gruppemedlem har læst op og fået styr på de emner, vi arbejder med.
4. Såfremt et gruppemedlem skulle droppe ud af studiet i løbet af projektet eller på anden vis indgå et længerevarende fravær, håber vi at personen vil give besked til gruppen snarest muligt, samt forsøge at overlever alle eventuelle noter, dokumenter, filer, billeder og andet, som har relevans til projektet og gruppen. Ydermere, vil gruppen indkalde til et statusmøde, hvor der tages status på nye opgaver og fordeling af arbejde baseret på manglende arbejdskraft fra det tidligere gruppemedlem.

Værktøjer & tekniske aftaler

1. Som udgangspunkt foregår al intern kommunikation gennem en fælles Messenger gruppe. Undtagelser kan forekomme ved online møder, hvor Discord bruges som standard og dermed vil visse beskeder også sendes via Discord.
2. Fildeling sker på to platforme. Skriftlige dokumenter og andre dokumenter vil gemmes, oprettes eller deles gennem en fælles google drive mappe. Undtagelser inkluderer, men er ikke begrænset til: Projektets kodefiler og gruppelog. Disse filer vil gemmes og deles gennem et fælles GitHub repository.
3. Kodestandarder såsom formateringer, navngivning af variable, funktioner, klasser, etc. aftales internt i gruppen, men følger som udgangspunkt industri-standarderne fremvist af undervisningen.
4. Gruppens medlemmer gør sig forpligtet til at kommentere sin kode til en grad, der gør det muligt for de andre gruppemedlemmer at læse og forstå koden uden yderligere forklaring om funktionalitet. Dette skal gøres regelmæssigt for at undgå eventuelt tidsspild ved et gruppemedlems fravær eller frafald fra uddannelsen.

Konflikthåndtering & eskalering

1. Hvis der opstår konflikter i gruppen, forsøger gruppen selv at løse dem.
2. Hvis det ikke lykkes, kan vejleder inddrages som neutral part for at hjælpe med at finde en løsning.

Deadlines

1. Uge 39 (23. September): Aflevering af samarbejdskontrakt og vejlederkontrakt.
2. Uge 41 (10. Oktober): Aflevering af projektgrundlag
3. Uge 46 (11. November): Aflevering af rapport samt kode - Iteration #1
4. Uge 49 (1. December): ProOnline deadline
5. Uge 50 (12. December): Endelig aflevering af kode
6. Uge 51 (19. December): Endelig aflevering af rapport
7. Uge 1-4: Eksamen

Benyttelse & revision af samarbejdsaftalen

1. Samarbejdsaftalen benyttes løbende gennem projektet.
2. Aftalen tages i brug ved konflikter eller andet relevant, til at lægge rammer for påkrævet handling.
3. Aftalen betragtes som udgangspunkt som statisk, men tages op til behandling ved hver faseafslutning: Hvad aftalte vi? Hvad gjorde vi? Hvad skal vi blive ved med at gøre? Hvad skal vi holde op med at gøre? Hvad skal vi begynde at gøre?

Underskrifter

Navn: **Dato:** **Underskrift:**
Aliza

Navn: **Dato:** **Underskrift:**
Mikkel Blom

Navn: **Dato:** **Underskrift:**
Octavian

Navn: **Dato:** **Underskrift:**
Yasser

Navn: **Dato:** **Underskrift:**
Ziad

10.2 Vejlederkontrakt

Dette er gruppens vejlederkontrakt:

Vejledningskontrakt

Dette dokument udgør en vejledningskontrakt for projektet i faget Semesterprojekt: Udvikling af Softwareprogrammer.

Formål

Formålet med kontrakten er at skabe klare rammer og forventninger til samarbejdet mellem projektgruppen og vejlederen. Kontrakten fastlægger roller, kommunikation og ansvar samt praktiske forhold som er til for at understøtte både projektets fremdrift og samarbejdet undervejs.

Projektinformation

Fagtitel: Semesterprojekt, udvikling af softwareprogrammer

Semester: 1 semester

Projektperiode: September 2025 - februar 2026

Projekttitel: Semesterprojekt

Rammer for samarbejdet

1. Agendaen, mødeindkaldelse og mødelokation fremsendes senest mandag før mødet.
2. Møder afholdes som udgangspunkt fysisk på campus, medmindre andet aftales.
3. Møderne afholdes fra kl. 08:00 til 08:45. Hvis dagsordenen er gennemgået, og begge parter vurderer, at mødet er afsluttet tilfredsstillende, kan mødet afsluttes tidligere.
4. Gruppen samler spørgsmål og problemstillinger, så de kan behandles under vejledningen.

Forventninger til vejledningen

1. Gruppen forventer at blive guidet gennem hele projektforløbet og eksamen.
2. Vejleder giver direkte og konkret feedback, både positiv og kritisk.
3. Vejleder møder til aftalt tid og sted.

4. Gruppen stiller forberedte spørgsmål og udnytter vejledningstiden effektivt.

Projektgruppens ansvar

1. At være den primære drivkraft i projektet
2. At planlægge møder med vejleder og fremsende dagsorden samt materiale i god tid.
3. At tage referat fra vejledermøder og dele det internt.
4. At stå for intern projektledelse, opgavestyring og konflikthåndtering.
5. At holde hinanden ansvarlige og sikre fremdrift.
6. At være åbne for feedback og bruge vejledningen aktivt.

Vejlederens ansvar

1. At forberede sig til møderne ved at gennemgå fremsendt materiale.
2. At stille kritiske og afklarende spørgsmål, der fremmer refleksion og faglig udvikling.
3. At give konstruktiv og ærlig feedback
4. At være tilgængelig for korte afklarende spørgsmål via mail/tlf mellem møderne.
5. At støtte gruppen i at holde fokus på målbeskrivelsen for *færdigheder, viden og kompetencer*¹.

Kommunikation

1. Kommunikation med vejledere foregår primært via mail.
2. Beskeder forventes besvaret inden for 2 hverdage
3. Hurtige, korte spørgsmål kan stilles uden dagsorden, mens mere komplekse problemstillinger behandles udelukkende til møder.

Afbud

1. Ved afbud fra vejleder forventes det, at få besked snarest muligt. Et nyt møde aftales i næste uge. (På nær ferier, helligdage og andre undtagelser)
2. Ved afbud fra gruppen meldes afbud senest 24 timer før mødet.

¹ Se kursusbeskrivelse her: <https://odin.sdu.dk/sitecore/index.php?a=fagbesk&id=157433&lang=da>

Konflikthåndtering

1. Hvis der opstår konflikter i gruppen, forsøger gruppen selv at løse dem.
2. Hvis det ikke lykkes, kan vejleder inddrages som neutral part for at hjælpe med at finde en løsning.

Tidsplan og milepæle

1. Gruppen udarbejder en overordnet tidsplan med milepæle (rapportudkast, kode leverancer, fremlæggelser osv.)
2. Vejleder orienteres løbende om status i forhold til tidsplanen.

Aftalens varighed

Denne aftale gælder fra projektets start og indtil den endelige eksamen er afsluttet. Aftalen kan justeres efter fælles overenskomst, hvis behov opstår.

Underskrifter

Gruppemedlemmer

Navn: **Dato:** **Underskrift:**

Aliza

Navn: **Dato:** **Underskrift:**

Mikkel Blom

Navn: **Dato:** **Underskrift:**

Octavian

Navn: **Dato:** **Underskrift:**

Yasser

Navn: **Dato:** **Underskrift:**

Ziad

Vejleder

Navn: **Dato:** **Underskrift:**

Jeppe

10.3 Gruppelog

Dette er gruppens gruppelog:

Gruppe Log for Semesterprojekt 2025

Indholdsfortegnelse

(SKABELON) REFERAT AF VEJLEDERMØDE	2
REFERAT AF INDLEDENDE SEMESTERPROJEKT MØDE	3
REFERAT AF VEJLEDERMØDE 23.09.2025	4
REFERAT AF VEJLEDERMØDE 30.09.2025	7
REFERAT AF VEJLEDERMØDE 03/10/2025	9
REFERAT AF GRUPPEMØDE 13/10/2025	10
REFERAT AF VEJLEDERMØDE 21/10/2025	11
REFERAT AF GRUPPEMØDE 25.10.2025	13
REFERAT AF VEJLEDERMØDE 28.10.2025	15
REFERAT AF GRUPPEMØDE 30.10.2025	16
REFERAT AF VEJLEDERMØDE 04.11.2025	17
REFERAT AF GRUPPEMØDE 14.11.2025	19
REFERAT AF VEJLEDERMØDE 18.11.2025	20
REFERAT AF VEJLEDERMØDE 21.11.2025	21
REFERAT AF VEJLEDERMØDE 25.11.2025	22
REFERAT AF GRUPPEMØDE 02.12.2025	23
REFERAT AF GRUPPEMØDE 11.12.2025	24
REFERAT AF VEJLEDERMØDE 16.12.2025	25
REFERAT AF GRUPPEMØDE 17.12.2025	27
REFERAT AF GRUPPEVEJLEDER 18.12.2025	28

(Skabelon) Referat af vejledermøde

1. Mødedetaljer

- Dato:
- Tidspunkt:
- Sted:
- Deltagere: Mikkel, Ziad, Octavian, Yaser, Aliza
- Fraværende:
- Referent:

2. Dagsorden

- Punkt

3. Gennemgang af punkter

- Vi fik gennemgået

4. Eventuelt

- eventuelt

5. Aftaler og deadlines

- Punkt

6. Næste møde

- Dato:
- Tidspunkt:
- Sted:
- Indhold:

Referat af indledende Semesterprojekt Møde

1. Mødedetaljer

- Dato: 16.09.2025 (Tirsdag)
- Tidspunkt: 11:00-13:00
- Sted: Lokale U156, SDU
- Deltagere: Alle gruppemedlemmer (ingen fraværende)
- Referent: Mikkel R. Blom

2. Dagsorden

- Lægge plan for udarbejdelse af gruppekontrakt
- Lægge plan for udarbejdelse af vejlederkontrakt

3. Gennemgang af punkter

- Gruppekontrakt: Gruppen lagde en plan for, hvordan gruppekontrakten skal udarbejdes og organiseres.
- Vejlederkontrakt: Gruppen lagde en plan for at udarbejde vejlederkontrakt.

4. Eventuelt

Vi snakkede om brug af LaTeX til opgaveskrivning. Vi besluttede os at gøre brug af LaTeX til den endelige formatering og opsætning af rapporten, men løbende vil vi udarbejde indholdet via Google Docs. Efterfølgende vil vi indsætte i LaTeX.

Ydermere besluttes det, at supplerende dokumenter såsom gruppekontrakt og andet, ikke behøver udarbejdes i LaTeX. Her benyttes bare Google Docs.

Til sidst, meldte Mathias sig som ansvarlig for, at vi får sendt invitation til vejleder fremover.

5. Aftaler og deadlines

- Mikkel R. Blom forbereder udarbejdet og finpudset gruppekontrakt til næste møde.
- Mathias forbereder udarbejdet og finpudset vejlederkontrakt til næste møde.

6. Næste møde

- Dato: 19.09.2025 (Fredag)
- Tidspunkt: 08:00-10:00
- Sted: Online
- Indhold: Gennemgang af de to udarbejdede kontrakter og indledende diskussion af problemformulering for projektet.

Referat af vejledermøde 23.09.2025

1. Mødedetaljer

- Dato: 23.09.2025 (Tirsdag)
- Tidspunkt: 8:00-13:00
- Sted: Lokale Ø22-612a-2, SDU
- Deltagere: Mikkel, Octavian, Aliza & Ziad.
- Fraværende: Yaser.
- Referent: Ziad

2. Dagsorden

- Velkommen og intro
- Gennemgang af vejlederkontrakt
- Gennemgang af gruppekontrakt
- Yderligere hjælp

3. Gennemgang af punkter

- Velkommen og intro:

Vejleder forklarer at det er vigtigt at gruppen gør en indsats til at kører møderne, fordi han ikke skal hive det ud af os. Den bedste måde skal gruppen selv finde ud af.

- Vejlederkontrakt:

Vi præsenterer vejledningskontrakt.

Vejleder kommenterer at andre vejleder ikke kommer til "at være tilgængelig for korte afklarende spørgsmål", og det er kun noget som han vil gå mod på. Det er ikke sikkert at andre vil.

Vejleder kommenterer at " *Den aflyste vejledning skal erstattes, så det samlede antal vejledninger ved forløbets afslutning svarer til det oprindeligt planlagte.* " ikke står fast, og det ikke er et krav.

Vejleder kommenterer at vi skal kontakte vejleder hurtigt, ved konflikthåndtering, så det ikke ender med at problemet bliver for stort og han ikke får mulighed for at stoppe det. Det skal så hurtigt som muligt op på gruppe-plan.

Kommentar er taget op fælles, og vejlederkontrakt er ændret.

- Gruppekontrakt:

Vi præsenterer gruppekontrakt.

Vejleder kommenterer omkring punktet, hvori der står *"gruppen stræber efter 12, men vil være tilfreds med 7."* og ville vide, hvordan vi betragter det og hvorfor vi skriver det.

Vejleder giver et tip, at det er vigtigt at forstå HVORFOR man gøre noget, og ikke kun læser op. Både til møderne men også til eksamen.

Vejleder kommenterer at kontrakten er en forventningsafstemning, og at der ikke er noget juridisk bindende. Det er et ønske til, hvad vi gerne vil have ud af det. Man skal forstå tanken bag det, "hvorfor må man ikke xyz" i stedet for bare "man må ikke xyz".

- Hvad skal vi i dag:

Kig på kontrakten med nye øjne. CHECK.

Arbejde på vores problemformulering, og bruge ingeniørmodellen til at finde noget som vi kan gøre. Vi skal allerede snakke om selve programmet nu.

Arbejde på problemanalysen, f.eks. validering af problemet, hvad er årsagen til problemet. Det skal laves i dag.

Næste uge starter problemanalyse processen, men dermed er det vigtigt at vi har lavet nogle tanker om det nu.

Vi mødes kl 8.00 næste uge, tirsdag. Vi skal ikke mødes fælles med de andre grupper.

Vi skal aftale, hvem der sender mail osv.

- Problemformulering:

Vi er på rette spor. Vejleder kommenterer at vi ikke får problemer med at finde problemformulering, fedt løft.

- Andre kommentar:

Vi må gerne lave noget GUI på programmet, men først efter halvvejssemester møde. Det vil sige indtil da må det kun være et konsolspil, og det er noget vi skal vurdere langs projektet.

4. Eventuelt

5. Aftaler og deadlines

6. Næste møde

- Dato: 25/09/2025
- Tidspunkt: torsdag KL 12 (Aflyst senere)
- Sted: U149D
- Indhold: Gennemgå spørgsmål og ting.
- Dirigent: Octavian
- Referent: Aliza

Referat af vejledermøde 30.09.2025

1. Mødedetaljer

- Dato: 30.09.2025
- Tidspunkt: 8:00-xx:xx
- Sted: Online
- Deltagere: Mikkel, Ziad, Octavian, Yaser
- Fraværende: Aliza
- Referent: Octavian

2. Dagsorden

- Kort præsentation af hvor langt vi er
- Valider vores ide + feedback
- Problemafgrænsning
- Næste skridt

5. Aftaler og deadlines

- Analysepunkter med deadline fredag d. 3.
 - Yaser: Hvad er problemet?
 - Mikkel: Problemtræ
 - Aliza: Målgruppeanalyse
 - Octavian: Aktøranalyse
 - Ziad: Begrundelse for afgrænsning
 - Punkterne gennemgås sammen fredag d. 3. kl. 10:00-12:00 fysisk på skolen (Lokale aftales senere efter Yaser booker lokale)
- Færdiggør problemanalyse til tirsdag d. 7.
- Færdiggør poster til tirsdag d. 7.

6. Næste møde

- Dato: d. 3. oktober 2025
- Tidspunkt: 10:00-12:00
- Sted: SDU (lokale kommer senere)
- Indhold: Gennemgang af det vi har lavet, samt en plan for udvikling af poster.

7. Referat

Hvorfor skriver vi det? Der er en masse belæg på, hvorfor vi har valgt en målgruppe på 10+ år. Hvorfor skriver vi det? Hvilken indflydelse har det på vores spil/produkt? Dette er spørgsmål vi skal tænke over og få på plads.

Måske er 10+ år en for bred målgruppe. Tænk over hvilke valg vi træffer – hvorfor skal det være en del af undervisningen? Er det bare fedt, eller er der en specifik grund. Det er en grundlæggende ide vi skal tænke over i alle aspekter af projektet: Hvorfor skriver vi det.

Er udvikling af et læringsspil vores problem? Er det ikke mere at folk ikke ved nok om affaldssortering? Vi skal tage et skridt tilbage og selv finde problemet. Normalt som ingeniør, får man stillet et problem, som man skal løse. Men i dette tilfælde, skal vi selv finde problemet.

Vi dykker ned i vores problemformulering og prøver at fokusere den mere omkring vores faktiske problemformulering. Vi omformulerer f.eks. ”gennem udvikling af et læringsspil” til ”gennem et læringsspil”. Her er tanken, at vores fokus er at viden skal øges mens man spiller spillet, og ikke gennem selve udviklingen af spillet.

Vi snakker om at ændre ”hvorfor” til bare ”kan” – så vi ikke skal fokusere rapporten på forskellige metoder på hvordan man kan. Men i stedet siger ”kan” vi løse det her problem gennem det her læringsspil. Så kan vi svare ja eller nej.

Det bliver også forstået at snakke om fail/learn eller trial/error princippet – og sige at vores problemformulering hedder noget i stilen med ”Kan vi, gennem trial/error princippet, øge viden om...”

Vi kan i problemanalysen sige, at vi er kommet frem til en problemformulering der lyder ”Hvordan kan vi...”, men så nævne, at ”vi i denne opgave har valgt en problemformulering hvor vi siger kan vi...”

Når vi beskriver vores afgrænsning, kan vi formulere det som ”De danske forhold” ifm. affaldssortering.

Hvad er omfanget af problemanalysen? Det er en god ide at finde ud af hvorfor man er forvirret og hvorfor det er relevant. F.eks. hvorfor skal pizzabakker med olie sorteres et specifikt sted.

Det skal gerne være så dybdegående, at vi til sidst, kan indsætte vores problemformulering og det giver mening med alt den viden vi har fundet. Slutningen af analysen skal gerne ende med ”Så derfor... er vores problemformulering...”

Hvad er årsagen? Hvad er konsekvensen? Hvorfor? – Problemtræ

Vi skal fremvise en A0 poster tirsdag d. 7., hvor vi præsenterer projektgrundlaget. Vi skal have samme overskrifter som eksemplet på itslearning. Vi skal gerne være meget kreative. Det hele handler om, hvordan vi kan præsentere noget viden.

Overstående ifm. møde med Jeppe. Herefter normalt gruppearbejde.

I dag skal vi lave ændringer til problemformulering for at få den specificeret. Starte på problemanalyse, problemtræ og poster. Yderligere skal vi genoverveje vores målgruppe.

Referat af vejledermøde 03/10/2025

1. Mødedetaljer

- Dato: 03.10.2025 (Fredag)
- Tidspunkt: 08:00-10:00
- Sted: Lokale ø22-612a-1, SDU
- Deltagere: Alle gruppemedlemmer (ingen fraværende)
- Referent: Octavian

2. Dagsorden

- Gennemgå analysen af: problemformulering, problemet og afgrænsninger, problemtræ, målgruppe, aktør og begrundelsen for afgrænsning.
- Begynd arbejde på poster for projektet.

3. Gennemgang af punkter

- Problemanalysen: Gruppen har fået gennemgået analysen for det overordnet problem.
- Poster: Gruppen har en grundlægende bedre forståelse af hvordan poster skal se ud og virke ud i verdenen.

4. Eventuelt

Vi snakkede om brug af LaTeX til opgaveskrivning. Vi besluttede os at gøre brug af Word indtil vi er færdig med rapporten hvor vi så vi gik og bruge tid på at konvertere det til LaTeX.

Vi har også brug for at bruge lidt ekstra tid i weekenden for at færdig gøre posteren.

Til sidst, tog vi en fælles beslutning om at det er dirigenten som er ansvarlig for at sende mail om vejleder mødet om tirsdagen.

5. Aftaler og deadlines

- Octavian laver aktøranalysen færdig i rapporten.
- Gruppen arbejder på posteren

6. Næste møde

- Dato: 04.10.2025 (Lørdag)
- Tidspunkt: 19:00-21:00
- Sted: Online
- Indhold: Gennemgang af hvad der mangler på posteren og i rapporten og færdig gøre begge til på tirsdag.

Referat af gruppemøde 13/10/2025

1. Mødedetaljer

- Dato: 13.10.2025 (Mandag)
- Tidspunkt: 11:00-15:00
- Sted: Lokale U149E, SDU
- Deltagere: Alle gruppemedlemmer (ingen fraværende)
- Referent: Aliza

2. Dagsorden

- Færdiggør noun/verb analysis
- Lave klasse/metode liste
- Lave indledende UML-diagram

3. Gennemgang af punkter

- Metoder: gruppen har gennemgået spillets vigtigste metoder
- Attributer: gruppen har gennemgået spillets vigtigste attributer

4. Eventuelt

Vi snakkede om hvorvidt det var relevant for os om at lave UML eller CRC. Vi besluttede kun at lave UML og finde ud af til næste vejledermøde om CRC er relevant.

5. Aftaler og deadlines

6. Næste møde

- Dato: 21.10.2025 (tirsdag)
- Tidspunkt: 08:00-13:00
- Sted: ø22-612a-1, SDU
- Indhold: Gennemgang af om CRC skal laves

Referat af vejledermøde 21/10/2025

1. Mødedetaljer

- Dato: 21.10.2025 (Tirsdag)
- Tidspunkt: 8:00-12:00
- Sted: ø22-612a-2, SDU
- Deltagere: Alle gruppemedlemmer (ingen fraværende)
- Referent: Yasser

2. Dagsorden

- Kort præsentation af færdig gjort ting
 - Noun/verb analyse
 - Krav (funktionel og ikke funktionel)
 - UML-diagrammer
- Spørgsmål til vejleder:
 - Skal vi både lave CRC og UML-diagrammer (svaret: Ja, mening med CRC er at man tager udgangspunkt i Noun/verb analysen. Rækkefølgen: Noun/Verb, CRC og så UML)
 - Skal vi lave UML-diagrammer for alle klasser i vores kode? (svaret: i rapporten tager man et stykke ud. Den vigtigste del. Hele UML-diagrammet kan man have som bilag)
 - Skal der sidehoved på rapporten og hvad skal den indeholde? (svaret: Ja det skal vi. Det er ikke et krav, dog en god ide at have med)
 - Hvilke kilde referencesystem skal vi bruge? (svaret: Vi skal finde noget der passer os. Der er intet krav omkring det.)
 - Skal vi inkludere et krav (Non-functional) omkring performance (svaret: Hvis vi synes at produktet har glæde af det så skal vi have det med som et krav)
- Vejledning til næste skridt i projektet
 - Flere krav til vores produkt/indhold
 - Have hele processen i rapporten
 - Lav CRC og UML færdigt. Overvej nedarvning fra de forskellige klasser
 - Begynde at brug UML-diagrammet til at lave vores spil
 - Trello: få opgaverne delt ud

3. Gennemgang af punkter

- CRC-kort
- Flere ikke-funktionelle krav
- Rapport

- VScode/live share og github commits

4. Eventuelt

5. Aftaler og deadlines

Vi har aftalt at bruge VScode til at kode sammen og benytte GitHub til at uploade den opdateret version af koden efterfølgende.

Vi mødes online på lørdag. Vi vil arbejde på følgende ting:

- Vi skal lave UML færdigt.
- Vi skal skrive videre på rapporten.
- Vi skal starte med at kode.
- Vi skal sætte Trello op

6. Næste møde

- Dato: 25.10.2025 (Lørdag)
- Tidspunkt: 08:00-12:00
- Sted: Online (Discord)
- Indhold:

Referat af gruppemøde 25.10.2025

1. Mødedetaljer

- Dato: 25.10.2025 (Lørdag)
- Tidspunkt: 11:00-13:00
- Sted: Online
- Deltagere: Aliza, Octavian, Mikkel, Ziad
- Fraværende: Yaser
- Referent: Mikkel

2. Dagsorden

- Påbegynd rapport for punkterne: Spilbeskrivelse, Noun/Verb analyse, CRC diagram og UML diagram
- Færdiggør UML diagram
- Påbegynd koden

3. Gennemgang af punkter

- Rapportpunkterne blev påbegyndt og en indledende tekst blev skrevet til alle punkterne fra dagsorden.
- UML diagrammet blev opdateret og udvidet til at indeholde alle relevante klasser og metoder fra CRC diagrammet, samt andre.
- Vi påbegyndte koden. Opsætte projektet og fordelte forskellige klasser fra UML diagrammet til hver gruppelem.

5. Aftaler og deadlines

Vi har aftalt at færdiggøre vores tilddelte kodelstykker, hver især, inden vejledermødet tirsdag. Kodelstykkerne blev fordelt ud fra UML diagrammet og lyder således:

Aliza: TrashObject

Octavian: InfoText, PlayerScore

Mikkel: Button

Ziad: Screen + Alle under-screens

Yaser: Trash, Trashcan

Vi mødes til vejledermøde Tirsdag, hvor Aliza er ordfører. Til mødet vil vi gennemgå følgende ting:

- Få feedback på CRC og UML diagram
- Få vejledning i næste skridt og hvordan der arbejdes sammen gennem kodeprocessen.
- Præsentation af koden hver især har lavet. Samt videreudvikling og sammensætning af kodelokkerne.

6. Næste møde

- Dato: 28.10.2025 (Tirsdag)
- Tidspunkt: 08:00-12:00
- Sted: SDU Lokale Ø22-612a-2

Referat af vejledermøde 28.10.2025

1. Mødedetaljer

- Dato: 28.10.2025 (Tirsdag)
- Tidspunkt: 08:00-12:00
- Sted: Ø22-612a-2
- Deltagere: Aliza, Octavian, Mikkel, Yasser, Ziad
- Fraværende: Ingen
- Referent: Octavian

2. Dagsorden

- - Få feedback på CRC og UML diagram
- - Få vejledning i næste skridt og hvordan der arbejdes sammen gennem kodeprocessen.

3. Gennemgang af punkter

- Vejleder skulle til tandlæge og aflyste mødet

5. Aftaler og deadlines

Vi vil udvikle koden og derfor har snakket om hvad der skal bygges videre på og delt det ud mellem gruppen på en måde hvor alle prøver at arbejde på hinanden kode for at få en bedre overblik over hvordan de andre koder, vi forventer at til næste gruppe møde har:

Alize fikset: PlayerScore

Mikkel videre bygget: Screen

Octavian begyndt at implimentere WoZ metode til levels

Yasser udviklet: TrashObject

Ziad ombyggede: InfoText

Vi mødes næste gang og ser hvor lang vi er nået og ser hvad vi skal i gang med der.

6. Næste møde

- Dato: 30.10.2025 (Torsdag)
- Tidspunkt: 12:00-14:00
- Sted: SDU Lokale Ø14-504a-3

Referat af gruppemøde 30.10.2025

1. Mødedetaljer

- Dato: 30.10.2025 (Torsdag)
- Tidspunkt: 11:00-13:00
- Sted: Ø14-504a-3
- Deltagere: Aliza, Octavian, Mikkel, Yasser, Ziad
- Fraværende: Ingen
- Referent: Octavian

2. Dagsorden

- Gennemgå kode.
- Snakke om de næste skridt i kode udviklingen

3. Gennemgang af punkter

- Gennemgået

5. Aftaler og deadlines

Til næste møde har vi uddelt det næste stykke arbejde:

Oprettes og fyldes alle slags Trash og Trashcans - Yaser

Oprettes og fuldes alle slags Tokens/Items - Mikkel

Laves LevelSelector + ItemLæsFunktion for alle rummene - Ziad

Gameplay/Gameloop: Liv, spawn trash, få det til at falde og bevæge sig ud fra inputs (HUSK NON BLOCKING). - Octavian

Infotext implementering + oprette generelle infotexter - Aliza

Vi mødes igen til vejledermøde.

6. Næste møde

- Dato: 04.11.2025 (Tirsdag)
- Tidspunkt: 08:00-xx:xx
- Sted: SDU Lokale Ø22-612a-2

Referat af vejledermøde 04.11.2025

1. Mødedetaljer

- Dato: 04.11.2025
- Tidspunkt: 8:00-16:00
- Sted: Ø22-612a-2
- Deltagere: Mikkel, Octavian, Yasser, Ziad
- Fraværende: Aliza
- Referent: Octavian

2. Dagsorden

- Give en kort præsentation af, hvor langt vi er i projektet og følge op på de ting vi har lavet siden si sås sidst.
- En opklaring af projektet/programmets krav og vores spil. Samt en vurdering af vores nye tilpasning af spillet.
- Tage et kig på de CRC kort og opdateret UML diagram vi har lavet.
- Få vejledning ift. De næste skridt i projektet og hjælp til at lægge en konkret plan til indhold og struktur af præsentation d. 11 november.

3. Gennemgang af punkter

Crc - vigtigt at forklare hvad man gøre og hvorfor, men husk rækkefølgen, crc også uml også kode, ikke vigtigt at navne hvilket man lavet først man hvilket skal laves først

UML - sensor vil spørger omkring klassernes nedrivning, hvad er endelige forskellen / hvorfor; eksempl - hvorfor lave forskellige klasser istedet for 1

Code - forskel på interface og nedrivning, for eksempel screens har en instans af moveButton() og ikke en nedrivning

Lave screen om fordi det ikke passer med beskrivelsen af hvad nedrivning gå ud på sammen med opbygningen af en klasse

Screen har samme struktur med forskellige indhold; ikke nedrivning

Efter koden er færdig kan vi lave en dokumentations uml, (det vi har til at starte med er en analysen uml (analyse klasse diagram))

Woz - ikke vigtigt for Jeppe, lige når vi starter spillet og viser knapper er vi væk fra Woz strukturen, der forventes ikke nogen form for genkendelse til woz strukturen, det vigtige er princippet bag funktionen af programmet

Forklare hvordan vi blev "inspireret af woz" som del af proces beskrivelsen, altså hvordan vi kom til vores kode "fra woz"

Præsentation - powerpoint, 15 min, gennemgå krav analyse design, det skal ud i en visning af programmet, vise programmet køre og snakke om de ting undervejs hvordan de reflektere i koden,

Vi skal vælge området vi vil have feedback til fra

Gennemgå hele processen altså hele flowet

4. Eventuelt

Pass

5. Aftaler og deadlines

Ingen

6. Næste møde

- Dato: 06.11.2025
- Tidspunkt: 12:00-14:00
- Sted: ???

Referat af gruppemøde 14.11.2025

1. Mødedetaljer

- Dato: 14.11.2025
- Tidspunkt: 10:00-15:00
- Sted: Ø20-611-1
- Deltagere: Mikkel, Ziad, Octavian, Yaser, Aliza
- Fraværende: Ingen
- Referent: Octavian

2. Dagsorden

- Snak om projektet, næste skridt, rapport og kode

3. Gennemgang af punkter

- Vi fik gennemgået og kodet en del sammen

4. Eventuelt

Idag har vi brugt alt tiden på at refinere koden, fixet bugs, vi skal dog stadig refrakturer den senere når den er helt færdig.

5. Aftaler og deadlines

- Vi har delt nogen kode tasks ud til næste møde
- Lås af rum - Mikkel
- Tilføj skrald - Octavian
- Level selector skal vise opdaget skrald - Ziad
- Gem opdaget skrald og forskellige highscore - Yaser
- Fald og opdag skrald - Aliza

6. Næste møde

- Dato: 18.11.2025
- Tidspunkt: 8:00-12:00
- Sted: Ø22-612-2

Referat af vejledermøde 18.11.2025

1. Mødedetaljer

- Dato: 18.11.2025
- Tidspunkt: 8:00 - 23:00
- Sted: Ø22-612a-2
- Deltagere: Mikkel, Ziad, Octavian, Yaser, Aliza
- Fraværende: Ingen
- Referent: Octavian

2. Dagsorden

- Afklaret kravspecifikation
- Gennemgå rapportens opbygning

3. Gennemgang af punkter

- Hvis en krav ikke passer kan man godt bare ændre det
- Det vi skriver er en teknisk rapport ikke en akademisk rapport. Forskellen er at i en akademisk rapport beskriver man emnet frem for projektet, det betyder at i vores opbygning skal vi beskrive vores problem, vores udførelse, vores løsning. Vi skal som bedst muligt forklare metoden, dermed forventes der heller ikke 100 sider, for eksempel er 20 meget passende.

4. Eventuelt

Idk

5. Aftaler og deadlines

- Folk skriver på rapporten
- Krav - Ziad
- Design - Mikkel
- Implimentering - Aliza
- Evaluering iteration 1 og 2- Octavian
- Proccesevaluering + unittest - Yasser

6. Næste møde

- Dato: 25.11.2025
- Tidspunkt: 8:00 - 12:00
- Sted: Ø22-612a-2

Referat af vejledermøde 21.11.2025

1. Mødedetaljer

- Dato: 20.11.2025
- Tidspunkt: 8:00 - 12:00
- Sted: Biblioteket
- Deltagere: Mikkel, Ziad, Octavian, Yaser, Aliza
- Fraværende: Ingen
- Referent: Aliza

2. Dagsorden

- Raffinere de sidste dele af koden, samt fixe bugs
- Gennemgået dele af koden

5. Aftaler og deadlines

- Folk skriver på rapporten
- Krav - Ziad
- Design - Mikkel
- Implimentering - Aliza
- Evaluering iteration 1 og 2- Octavian
- Proccesevaluering + unittest - Yasser

6. Næste møde

- Dato: 25.11.2025
- Tidspunkt: 8:00 - 12:00
- Sted: Ø22-612a-2

Referat af vejledermøde 25.11.2025

1. Mødedetaljer

- Dato: 25.11.2025
- Tidspunkt: 8:00-13:30
- Sted: Projektrum
- Deltagere: Mikkel, Ziad, Octavian, Aliza, Yaser
- Fraværende: Ingen
- Referent: Ziad

2. Dagsorden

- Give en kort præsentation af, hvor langt vi er i projektet.
- Stille spørgsmål ift. Projektarbejde, b.la. Spørgsmålene herunder:
 - Hvor skal vi skrive om unittest?
 - I vores lagdeling. Er presentation laget ikke teknisk set KUN ConsoleDrawer? Fordi det er den eneste der FAKTISK skriver til skærmen. Alle andre scripts kalder ConsoleDrawer og får den til at skrive?
 - Hvad er kontrolskema for portefølje?
 - Andre Spørgsmål ift. opsætning af rapporten
- Få vejledning ift. de næste skridt i projektet og evt. få input til krav og forslag til problemanalysen.
- Uddele opgaver til næste møde.

3. Gennemgang af punkter

- Vi fik gennemgået de adskillige spørgsmål med Jeppe, og fik dem rettet til, så det er korrekt.
- Vi uddelegerede de kommende opgaver, der skulle være færdige til næste møde.

4. Eventuelt

5. Aftaler og deadlines

- Færdiggørelse af alle vores emner til rapporten, før næste møde.

6. Næste møde

- Dato: 2/12/2025
- Tidspunkt: 8:00
- Sted: Projektrum
- Indhold: Gennemgå rapporten og nye ændringer, uddele nye opgaver.

Referat af gruppemøde 02.12.2025

1. Mødedetaljer

- Dato: 02.12.2025
- Tidspunkt: 09:00-10:00
- Sted: Projektrum
- Deltagere: Aliza, Mikkel, Ziad, Octavian, Yaser
- Fraværende: Ingen
- Referent: Ziad

2. Dagsorden

- Gennemgå rapporten og nye ændringer.
- Uddele nye opgaver.

3. Gennemgang af punkter

- Vi mødtes uden vejleder denne gang, da der ikke var behov.
- Vi fik gennemgået rapporten og de nye ændringer der blev lavet til denne gang.
- Vi gennemgik dele af koden, og rettede småfejl til m.m.
- Vi aftalte næste møde.

4. Eventuelt

5. Aftaler og deadlines

- Vi aftalte at mødes igen dagen efter BA-MAT1 eksamen, for at gennemgå vores kode og aflevere det sammen.

6. Næste møde

- Dato: 11/12/2025
- Tidspunkt: 12:00
- Sted: Vi finder ud af det
- Indhold: Endelig gennemgang af koden og aflevering.

Referat af gruppemøde 11.12.2025

1. Mødedetaljer

- Dato: 11.12.2025
- Tidspunkt: 8:00-10:00
- Sted: Online
- Deltagere: Mikkel, Ziad, Octavian, Aliza, Yaser
- Fraværende: Ingen.
- Referent: Ziad

2. Dagsorden

- Gennemgang af vores egne noter til koden, samt en fælles gennemgang af dele af koden.
- Aflevering af koden.
- Uddele opgaver til næste gang

3. Gennemgang af punkter

- Vi fik gennemgået alles noter til koden, som der var blevet lavet som lektie, og fik opklaret alle dele.
- Vi fik afleveret koden.

4. Eventuelt

5. Aftaler og deadlines

- Vi fik opdelt og uddelt de forskellige opgaver, der skulle laves til næste gang.

6. Næste møde

- Dato: 16/12/2025
- Tidspunkt: 08:00
- Sted: Projektrum
- Indhold: Møde med vejleder, og uddele opgaver til næste gang.

Referat af vejledermøde 16.12.2025

1. Mødedetaljer

- Dato: 16.12.2025
- Tidspunkt: 8:00-10:00
- Sted: Online med vejleder, gruppe i projektrum
- Deltagere: Mikkel, Ziad, Octavian, Aliza
- Fraværende: Yasser
- Referent: Ziad

2. Dagsorden

- Møde med vejleder
- Uddele opgaver.

3. Gennemgang af punkter

- Vi gik igennem målbeskrivelserne under "projektaflevering og eksamen.pptx", og snakket om, hvad vi skal kunne vise til eksamen. Det er disse punkter vi kommer til at snakke om, ved eksamen. Så vil de spørge b.la. til disse, og relatere det til vores rapport, og spørge ind til, hvor vi har de forskellige ting og hvordan det står osv.
- Derudover gik vi igennem resten af dokumentet.
- Vi skal huske at respektere vores medier, og sørge for at de kan ses, hvis vi printer det ud på papir.
- Gruppefremlæggelsen er en præsentation af projektet, hvor vi skal vise software, vores arbejde, osv. Vi skal fremlægge projektet og produktet. Vi skal ikke fremlægge vores rapport, det gøre vi ved den individuelle del.
- Vejleder nævner at vi skal huske at læse opgaven op, og se om det er letlæseligt. Det er et vigtigt punkt ift. Rapporten. Derudover nævner vejleder at vi alle skal kunne alle dele af rapporten, projektet, og selve præsentationen.

4. Eventuelt

- Vejleder mener at eksamen umiddelbart kommer til at være tirsdag d. 13/01/2026.

5. Aftaler og deadlines

- Alle skal have læst rapporten igennem, og skrevet, hvad der skal arbejdes på og forbedres. Så mødes vi igen, og går det hele igennem.

6. Næste møde

- Dato: 17/12/2025

- Tidspunkt: 19:45
- Sted: Online
- Indhold: Gennemgå vores individuelle tanker om rapporten, og rette den til. Derudover at læse det hele igennem sammen, og forbedre den yderligere.

Referat af gruppemøde 17.12.2025

1. Mødedetaljer

- Dato: 17.12.2025
- Tidspunkt: 19:45-01:45
- Sted: Online
- Deltagere: Mikkel, Ziad, Octavian, Yaser, Aliza
- Fraværende: Ingen.
- Referent: Ziad

2. Dagsorden

- Gennemgå vores individuelle tanker om rapporten, og rette den til.
- Derudover at læse det hele igennem sammen, og forbedre den yderligere.

3. Gennemgang af punkter

- Vi fik gennemgået størstedelen af rapporten, men da det blev trætte, valgte vi at stoppe for i dag, og arbejde videre dagen efter.

4. Eventuelt

5. Aftaler og deadlines

6. Næste møde

- Dato: 18/12/2025
- Tidspunkt: 12:00
- Sted: Online
- Indhold: Færdiggørelse af rapporten og aflevering.

Referat af gruppemøde 18.12.2025

1. Mødedetaljer

- Dato: 18.12.2025
- Tidspunkt: 12:00-15:00
- Sted: Online
- Deltagere: Mikkel, Ziad, Octavian, Aliza
- Fraværende: Yaser
- Referent: Ziad

2. Dagsorden

- Færdiggøre rapporten.
- Aflever rapporten.

3. Gennemgang af punkter

- Vi fik gennemgået alle punkter vi havde skrevet op til forbedringer, og har afleveret rapporten.

4. Eventuelt

5. Aftaler og deadlines

6. Næste møde

- Dato:
- Tidspunkt:
- Sted:
- Indhold:

10.4 Noun/Verb Analyse

Herunder fremgår den fulde Noun/Verb analyse af vores programbeskrivelse.

Noun/verb analysis

Menu:

Når spillet åbnes, skal en menu-skærmen vises.

Menu-skærmen skal indeholde 4 knapper:

1. "PLAY" som fjerner menu-skærmen og starter spilcyklus.
2. "OVERSIGHT"-knap der fjerner menuen og starter oversigt-sidens.
3. "CREDITS" knap, der fjerner menuen og starter credits-siden.
4. "AFSLUT" knap der lukker spillet.

"PLAY" knappen er markeret til at starte med. Spilleren kan skifte markering med op/ned piletasterne og vælge en knap med "enter".

Oversigt-siden:

Indeholder en liste med skraldespande, en til hver af de fem skraldespande. Når man markerer en af skraldespandene, understreges den individuelle skraldespands information.

Skraldespand informationsside:

Her kan spilleren se, hvilket skrald hører til i det valgte skraldespand.

Credits-siden:

Her kan spilleren se en liste over udviklere og deres roller.

Spilcyklus:

I spil-cyklussen starter spilleren med 3 liv, og spillet slutter, når spilleren ikke har flere liv. Imens spilleren har liv tilbage, skal skrald "spawne" fra toppen af skærmen og falde ned mod 5 skraldespande. Skraldet skal styres fra side til side med piletasterne. Når skraldet er faldet ned til skraldespandene, skal skraldet slettes og der skal tjekkes om skraldet ramte den korrekte skraldespand. Hvis ja, øges scoren, og spillet fortsætter. Hvis nej, skal spillet pauses og der vises en infotekst omkring hvorfor det var sorteret forkert og hvordan det gøres korrekt næste gang. Ydermere, mister spilleren et liv. Spillet fortsætter indtil spilleren ikke har flere liv, og hastigheden øges gradvist over tid. Skraldet skal repræsenteres af ét ikon, hvor selve typen af skrald skal skrives øverst i et hjørnet af skærmen.

Infotekst (Aliza):

Når skraldet lander i den forkerte skraldespand, vil der vises en infotekst frem i et lille vindue midt på skærmen. I den tekst vil der stå, hvor skraldet rigtigt skal være, og hvorfor skraldet skal være der. Når spilleren har læst dette, kan personen klikke dette vindue væk oppe i et kryds i højre hjørne. Spilleren kan nu prøve igen.

Loose screen/end screen:

Når du har sorteret forkert flere gange og ikke har flere liv, kommer der en "Game over/end screen" op. Spillerens highscore bliver gemt i en liste. På skærmen vil der stå "Game over"-tekst på dansk. Udover det vil der være nogen knapper, herunder:

- Restart knap. Trykker man på denne knap vil man kunne starte et nyt runde op
- Return to menu knap. Her kan man gå tilbage til start menu.
- Et "leaderboard" knap. Trykker man på denne knap vil man kunne se et "leaderboard" med de seneste højeste score

Highscore system:

Pointene kan ses oppe i venstre hjørne i en boks, og forøges hver gang skraldet rammer den rigtige skraldespand.

Herunder fremgår den fulde Noun/Verb liste, før filtrering.

Herunder fremgår den fulde Noun/Verb liste

Verb	Noun
Åbne	Spil
Vise	Menu
Indeholde	Skærm
Fjerne	Spilcyklus
Start	Knap
Luk	Oversigt
Markere	Side
Kan	Credits
Skift	
Vælge	Markering
Indeholde	Piletaster
Understrege	Liste
Kan	Skraldespan
Se	Skrald
Høre	Udviklere
Spawne	Liv
Falde	Scoren
Slette	Infotekst
Tjekke	Hastigheden
Øge	Tid
Pause	Ikon
Gøre	Type
Mister	Hjørnet
Fortsætter	Vindue
Repræsentere	Tekst
Skrive	Kryds
Lande	Game Over / End Screen
Være	Leaderboard
Læst	Point
Klikke	Boks
Sorteret	
Kommer	
Bliver	
Gemt	
Stå	
Restart	
Trykker	
Starte	
Gå	

Herunder fremgår den fulde filtreret Noun/Verb liste

Verb (Potentielle metoder)	Noun (Potentielle klasser)
Åbne	Spil
Vise	Menu
Fjerne	Knap
Start	Side
Luk	Skraldespand
Markere	Skrald
Skifte	Infotekst
Vælge	Leaderboard
Spawne	
Falde	
Slette	
Tjekke	
Pause	
Miste	
Fortsætter	
Skrive	
Lande	
Læse	
Klikke	
Sorteret	
Gemt	
Stå	
Restart	
Rammer	

10.5 Samlede CRC-Kort

Herunder fremgår de fulde CRC-kort for alle systemets klasser fra #1 Iteration.

Game	
Ansvar	Samarbejde
■ Holder styr på game state ■ Holder styr på score og liv ■ Styrer visning på skærmen ■ Styrer hvilket skrald der falder ■ Kører "gameloop": - Input fra spiller - Pause/Afslut - Øge sværhedsgrad	- TrashObject - Screen - Trashcan - PlayerScore - InfoText

Screen	
Ansvar	Samarbejde
■ Skal kunne fremvise forskellige menuskærme ■ Skal kunne fremvise spilskærm ■ Skal kunne håndtere interaktion med knapper	- Game - Button

Trash	
Ansvar	Samarbejde
■ Holder styr på skraldets navn ■ Holder styr på hvilken skraldespand er den rigtige ■ Kan sende navn og korrekt skraldespand	- Trashcan - TrashObject

TrashObject

Ansvar	Samarbejde
■ Skat kunne rykke position ud fra spiller input ■ Skat kunne holde styr på typen af skrald ■ Skat holde styr på dens position ■ Skat kunne dannes/initialiseres ■ Skat kunne falde ■ Skat kunne tjekke om den har ramt den rigtige skraldespand	- Trash - Trashcan - Game

Trashcan

Ansvar	Samarbejde
■ Skat indeholde en liste af alle korrekt stykker Trash til den pågældende skraldespand ■ Skat kunne tjekke om et stykke skrald er i den liste	- Trash - TrashObject - Game - Screen

Button

Ansvar	Samarbejde
■ Skat kunne markeres ■ Skat kunne åbne menuskærme/spil	- Screen

InfoText

Ansvar	Samarbejde
■ Skat kunne registrere/kontrollere interaktion (at det bliver læst) ■ Skat kunne åbnes og lukkes ■ Skat kunne vise noget tekst	- Trash

PlayerScore

Ansvar	Samarbejde
■ Opbevarer spillerens score ■ Kan forøge spilleren score ■ Kan nulstille scoren ■ Skal kunne sende den nuværende score ■ Skal kunne finde og gemme highscoren på computeren	- Game

10.6 Kravspecifikationer

Herunder fremgår de fulde kravspecifikationer fra #1 og #2 Iteration.

Tabel 7: #1 Iteration: Funktionelle Krav

Id	Navn	Beskrivelse	Status
F1.01	Vis menu-skærme	Programmet skal kunne vise flere forskellige menu-skærme.	Færdiggjort
F1.02	Vis tekst & infobokse	Programmet skal kunne vise tekst og forskellige infobokse.	Færdiggjort
F1.03	Gameloop	Programmet skal have spil-loop, så spilleren kan spille uendeligt. Ydermere skal spillet gradvist blive sværere (hurtigere).	Færdiggjort
F1.04	Start & afslut spil	Programmet skal kunne startes og afsluttes.	Færdiggjort
F1.05	Objekt håndtering	Programmet skal kunne håndtere objekter, herunder: spawn, falde, kontrollere, osv.	Færdiggjort
F1.06	Tjek affald	Programmet skal kunne identificere om skrald er sorteret i den rigtige skraldespand.	Færdiggjort
F1.07	Score system	Programmet skal kunne gemme spillernes score, gemme highscore, osv. Programmet skal kunne genkende den højeste score.	Færdiggjort
F1.08	Liv system	Programmet skal kunne håndtere spillerens "liv", herunder: hvor mange liv spilleren har, om det er "game over" osv.	Færdiggjort
F1.09	Tastatur-input	Programmet skal tage input fra tastaturet.	Færdiggjort

Tabel 8: #1 Iteration: Non-Funktionelle Krav

Id	Navn	Beskrivelse	Status
NF1.01	Kompatibilitet	Programmet skal kunne køres på en standard Windows computer.	Færdiggjort
NF1.02	Brugervenlighed	Programmet skal kunne være nemt at bruge for unge i folkeskole.	Færdiggjort
NF1.03	Læringsorienteret	Spilleren af programmet skal kunne lære noget omkring affaldssortering ved brug af spillet i en længere, uspecificeret periode.	Færdiggjort
NF1.04	Ydeevne	Programmet skal køre smoothly.	Færdiggjort
NF1.05	Læsbarhed	Teksten skal være læsbar, og størrelsen skal være passende.	Færdiggjort
NF1.06	Vedligeholdelsesvenlighed	Kodebasen skal være veldokumenteret og let at udvide med nye funktioner.	Færdiggjort
NF1.07	Felt tolerance	Programmet skal kunne håndtere ugyldigt input uden at crashe.	Delvist færdiggjort
NF1.08	Lokalisering	Alle tekster i programmet skal være på dansk og følge retskrivningsreglerne.	Færdiggjort
NF1.09	Sikkerhed	Programmet må ikke indsamle eller gemme persondata om brugerne.	Færdiggjort
NF1.10	Grafik	Grafiske elementer skal være klare og intuitive for målgruppen.	Færdiggjort

Tabel 9: #2 Iteration: Nye Funktionelle Krav

Id	Navn	Beskrivelse	Status
F2.01	Fejlfri program	Ingen tydelig fejl og mangler på kørende program. Her mener vi, at der ikke er noget der skal hindre brugerens spiloplevelse og/eller afvige dem fra spillets gang (bugs).	Delvist færdiggjort
F2.02	Lås af rum	De 5 forskellige rum (udover start rummet) skal låses indtil alle typer skrald fra det tidligere rum er fundet.	Færdiggjort
F2.03	Tilføj skrald	Forskellige typer skrald skal indsættes i hvert rum, så spilleren har mulighed for at opdage nye slags skrald fra rum til rum.	Færdiggjort
F2.04	Level selector	Level selectoren skal vise hvilke slags skrald er opdaget for hvert rum.	Færdiggjort
F2.05	Gem og hent opdaget skrald og highscore	Spillet skal kunne gemme og hente forskellige highscores for hvert rum, samt gemme hvilke slags skrald er opdaget for hvert rum.	Færdiggjort
F2.06	Fald og opdag skrald	Nyt skrald fra hvert rum skal gradvis indføres gennem spillets gang – så skrald langsomt opdages i intervaller, frem for alt på en gang.	Færdiggjort

10.7 Kontrolskema for Rapport

Skemaet udfyldes ved i kolonnen “Opfyldt” at markere de krav, der menes at være opfyldt med et + og de uopfyldte med et -.

Kapitel	Indhold	Opfyldt +/-
Forside	Projekttitel, uddannelsesinstitution, fakultet, institut, uddannelse, semester, kursuskode, projektperiode, vejleder, projektgruppe og projektdeltagere (fornavn, efternavn, sdu-email). Må gerne have illustrationer.	+
Indholdsfortegnelse	Samlet indholdsfortegnelse for hele rapporten.	+
Indledning	• Projektets rammer og baggrunden for projektet. • Problemanalysen. • Redegørelse for det udleverede framework. • Problemformulering og afgrænsninger. • Metoder. • Tidsplanen.	+
Krav	De samlede krav (modtagne og formuleret af jer). Analyse af kravene, herunder: • Noun-Verb analyse • CRC kort	+
Design	Beskrivelsen af designet, herunder et eller flere UML-klassediagrammer. I beskrivelsen indgår beskrivelse af arkitektur og brugergrænsefladedesign.	+
Implementering	Beskrivelse af implementering med centrale dele af programkoden.	+
Evaluerings	Evaluerings af om løsningen lever op til kravene, fx gennem en brugerundersøgelse eller brugerafprøvning. Beskrivelse af hvad der er opnået og hvad der ikke er opnået i projektet i forhold til det – som i indledningen beskrevet – forventede. Beskrivelse af styrker og svagheder ved resultaterne og om der kunne være opnået bedre resultater. Beskrivelse af kendte fejl.	+
Iteration #1 og #2	Er der i ovenstående afsnit (Krav, design, Implementering og Evaluerings) medtaget resultater både fra iteration #1 og fra iteration #2?	+
Konklusion	Opsummering af resultaterne og evalueringen. Svar på problemformuleringen.	+
Procesevaluering	Processen og gruppens refleksion over processen: Læringsprocessen, teamroller, samarbejdet internt i gruppen og med vejleder, projektarbejdsformen, arbejdsformer, metoder, skriveprocessen, den tidsmæssige styring af projektet, ledelse af projektet, arbejdsfordeling i projektet m.m. Bemærk: I skal have et underafsnit kaldet “Generativ AI” hvori I forklarer hvordan I har brugt generativ AI i projektet.	+
Samarbejdsaftale	Er samarbejdsaftalen indsat?	+
Vejlederaftale	Er vejlederaftalen indsat?	+
Projektlog	Er der indsat adresse på og et link til projektloggen?	+
Kontrolskema	Er dette kontrolskema blevet udfyldt?	+